Geometrical Parameters in Isogeometric Analysis and Reduced Order Modeling for Structural Dynamics

Master 2 Thesis

Mumen Wehbe

Supervisor: Christophe Hoareau

August 25, 2026

Abstract

Modern engineering design, structural health monitoring, and the realization of interactive web applicationss require high-fidelity numerical models that can be evaluated in real-time. In structural dynamics, resolving transient responses under varying parameter settings typically relies on the Finite Element Method (FEM). However, high-fidelity FEM simulations of structural components demand massive computational resources and time-consuming remeshing, preventing interactive design exploration or real-time control.

To address these challenges, this thesis investigates a real-time parametric simulation framework founded on three core pillars: **Isogeometric Analysis (IGA)**, **Reference Domain Pullback Mapping**, and **Energy-Conserving Sampling and Weighting (ECSW)** hyper-reduction. By using Non-Uniform Rational B-Splines (NURBS) as basis functions for both geometry representation and field approximation, IGA eliminates geometric approximation errors. Concurrently, projection-based Reduced Order Modeling (ROM) projects high-dimensional systems onto low-dimensional subspaces.

A major contribution of this work is resolving the vector space incompatibility caused by shape parameter variations. By introducing a smooth geometric pullback mapping from parameter-dependent physical domains to a fixed reference configuration $\hat{\Omega}$, snapshot vectors remain structurally invariant across parameter updates. To bypass full-mesh assembly online, the ECSW hyper-reduction scheme selects a minimal sparse element subset E^*.

Developed during an internship, the ultimate goal of this project was to design an interactive, real-time application for educational purposes, enabling teachers and students to run transient dynamic simulations effortlessly. The resulting IGA-Pullback-ECSW framework is deployed in a client-side WebGL application, achieving sub-millisecond evaluation times ($< 1.5\,\mathrm{ms}$, $\geq 60\,\mathrm{FPS}$), a **64.5× speedup** over the Full Order Model, and relative L_2 displacement errors under 0.054%.

Résumé

La conception en ingénierie moderne, la surveillance de la santé des structures et la réalisation de jumeaux numériques nécessitent des modèles numériques de haute fidélité pouvant être évalués en temps réel. En dynamique des structures, la résolution des réponses transitoires sous diverses configurations paramétriques repose généralement sur la méthode des éléments finis (MEF). Cependant, les simulations MEF de haute fidélité des composants structuraux exigent des ressources de calcul massives et un remaillage chronophage, empêchant l'exploration interactive de conception ou le contrôle en temps réel.

Pour relever ces défis, cette thèse étudie un cadre de simulation paramétrique en temps réel fondé sur trois piliers fondamentaux : **l'Analyse Isogéométrique (IGA)**, **le Transport en Arrière (Pullback) vers un Domaine de Référence**, et l'hyper-réduction par **Échantillonnage et Pondération Conservant l'Énergie (ECSW)**. En utilisant des B-splines rationnelles non uniformes (NURBS) comme fonctions de forme à la fois pour la représentation géométrique et l'approximation des champs, l'IGA élimine les erreurs d'approximation géométrique. Conjointement, la réduction de modèle basée sur la projection (ROM) projette les systèmes à haute dimension sur des sous-espaces de faible dimension.

Une contribution majeure de ce travail est la résolution de l'incompatibilité des espaces vectoriels causée par les variations des paramètres de forme. En introduisant une application géométrique de pullback lisse reliant les domaines physiques dépendants des paramètres à une configuration de référence fixe $\hat{\Omega}$, les vecteurs de snapshots restent structurellement invariants lors des mises à jour des paramètres. Pour contourner l'assemblage complet du maillage en ligne, le schéma d'hyper-réduction ECSW sélectionne un sous-ensemble minimal d'éléments actifs E^*.

Développé dans le cadre d'un stage, l'objectif ultime de ce projet était de concevoir une application interactive et en temps réel à des fins pédagogiques, permettant aux enseignants et aux étudiants d'exécuter des simulations dynamiques transitoires sans effort. Le cadre IGA-Pullback-ECSW en résultant est déployé dans un Jumeau Numérique WebGL interactif côté client, atteignant des temps d'évaluation inférieurs à la milliseconde ($< 1.5\,\text{ms}$, $\geq 60\,\text{FPS}$), une **accélération de $\mathbf{64.5}\times$** par rapport au modèle d'ordre complet, et des erreurs relatives L_2 sur le déplacement inférieures à 0.054%.

Acknowledgements

I express my deepest gratitude to my supervisor, **Christophe Hoareau**, for his guidance, insightful discussions, and continuous support throughout this Master 2 research project.

I also extend my sincere appreciation to the academic staff and research teams at Conservatoire National des Arts et Métiers (CNAM) for providing an inspiring environment and excellent computational facilities. In particular, I would like to thank Jean-François Deü for his significant contributions, Mireille Gillet and Cécile for handling the administrative parts, and Xavier Amandoleses who went in front of HR to make this internship possible. Special thanks also go to Pr. Rouleau, Pr. Aucejo, and Pr. Laurent for their support.

Contents

List of Figures

List of Tables

List of Algorithms

Nomenclature

The primary mathematical symbols, coordinate systems, and operators utilized throughout this thesis are defined below:

Geometric and Parametric Spaces

$\Omega(\boldsymbol{\mu})$	Parameterized physical domain of the notched cantilever beam
$\hat{\Omega}$	Parameter-independent reference domain
$\hat{\Omega}_{\text{param}}$	Parametric patch domain ($[0,1]^2$)
$\tilde{\Omega}$	Standard parent element domain ($[-1,1]^2$) for Gauss quadrature
Γ_D, Γ_N	Dirichlet and Neumann boundary zones in physical coordinates
$\hat{\Gamma}_D, \hat{\Gamma}_N$	Dirichlet and Neumann boundary zones mapped to the reference domain
$\tilde{\boldsymbol{\xi}} = (\tilde{\xi}, \tilde{\eta})^T$	Parent element coordinates in $[-1,1]^2$
$\boldsymbol{\xi} = (\xi, \eta)^T$	Parametric coordinates in the patch domain $\hat{\Omega}_{\text{param}}$
$\hat{\mathbf{x}} = (\hat{x}, \hat{y})^T$	Spatial coordinates in the reference domain $\hat{\Omega}$
$\boldsymbol{x} = (x, y)^T$	Spatial coordinates in the physical domain $\Omega(\boldsymbol{\mu})$
$\boldsymbol{\Psi}$	Diffeomorphic geometric mapping $\hat{\Omega} \to \Omega(\boldsymbol{\mu})$
$\mathbf{d}(\boldsymbol{\mu})$	Parameter-dependent control point displacement field
$\mathbf{J}_{\boldsymbol{\xi}}$	IGA coordinate Jacobian matrix ($\frac{\partial \mathbf{x}}{\partial \boldsymbol{\xi}}$)
$\mathbf{J}_{\hat{\mathbf{x}}}$	Continuum pullback Jacobian matrix ($\frac{\partial \boldsymbol{\Psi}}{\partial \hat{\mathbf{x}}}$)
$J_{\boldsymbol{\xi},\boldsymbol{\mu}}$	Determinant of the coordinate Jacobian matrix $\mathbf{J}_{\boldsymbol{\xi}}$
$\boldsymbol{\mu} = (r, x_c)^T$	Geometric design parameter vector (notch radius, center location)

Isogeometric Analysis (IGA) Discretization

$\Xi, \mathcal{H}$	Knot vectors defining the B-spline parameter spaces
p, q	Polynomial degrees of the B-spline basis functions in ξ and η directions
$N_{i,p}, M_{j,q}$	Univariate B-spline basis functions
$R_{i,j}^{p,q}$	Bivariate NURBS rational basis functions
$w_{i,j}$	Projective control weights corresponding to control nodes
$\mathbf{P}_{i,j}$	B-spline/NURBS control points forming the physical geometry mesh lattice
$\mathbf{u} = [u, v]^T$	Discretized displacement field vector
$\mathbf{U}$	Vector of physical nodal degrees of freedom
$\mathbf{B}$	Strain-displacement gradient matrix
$\mathbf{D}$	Linear elastic constitutive matrix under plane stress assumptions

Reduced Order Modeling (ROM) & Hyper-Reduction

N	Number of degrees of freedom in the Full Order Model (FOM)
r	Dimension of the reduced coordinate subspace ($r \ll N$)
$\mathbf{\Phi}$	Orthonormal reduced basis projection matrix ($\mathbb{R}^{N \times r}$)
$\mathbf{q}$	Vector of generalized reduced coordinates ($\mathbb{R}^r$)
$\hat{\mathbf{M}}, \hat{\mathbf{K}}$	Reference-mapped (pulled-back) global mass and stiffness matrices
$\mathbf{M}_r, \mathbf{K}_r$	Projected reduced-order mass and stiffness matrices
E^*	Number of active selected ECSW elements ($E^* = 28$)
ϵ_{tol}	Non-Negative Least Squares (NNLS) convergence residual tolerance (1×10^{-4})
Ω_{ECSW}	Reduced integration domain consisting of ECSW active elements
$\mathbf{w} = [w_e]^T$	Non-negative weights associated with ECSW active elements
ω_n	Natural frequency of vibration (rad/s)
$\mathbf{F}_{\text{int}}, \mathbf{F}_{\text{ext}}$	Internal elastic restoring and external load vectors

Chapter 1

Introduction & State of the Art

1.1 Introduction

Modern engineering design, structural health monitoring, and the realization of interactive web applicationss[1] necessitate a fast numerical evaluation depending on the application (real-time, uncertainties, parametric optimization, etc.). In structural dynamics, resolving the transient response of complex geometries under varying parameter settings typically relies on the Finite Element Method (FEM). However, high-dimensional FEM simulations of structural components demand important computational resources and time-consuming remeshing.

To address these challenges, this master thesis combines two powerful numerical paradigms: **Isogeometric Analysis (IGA)** and projection-based **Reduced Order Modeling (ROM)**. By using Non-Uniform Rational B-Splines (NURBS) as the basis functions for both geometry representation and field approximation, IGA eliminates geometric approximation errors. Concurrently, ROM projects the high-dimensional full-order systems onto low-dimensional subspaces in order to accelerate the computational times.

1.2 Key Concepts of the Study

Before diving into variational continuum mechanics, the core numerical methods of this framework can be understood through three main categories:

- **Isogeometric Analysis (IGA)**: Traditional Finite Element Analysis breaks curved geometry into straight segments, resulting in geometric approximation errors. Isogeometric Analysis retains exact CAD spline curves (NURBS) directly, eliminating discretization errors.

- **Reduced Order Modeling (ROM)**: Full-order simulations calculate thousands of degrees of freedom per time step. Projection-based Reduced Order Modeling via Proper Orthogonal Decomposition (POD) compresses these high-dimensional fields into a low-dimensional subspace of dominant mode shapes, capturing over 99.99% of the system energy.

- **Hyper-Reduction (ECSW)**: When geometrical parameters are non-affine (no separation of operators and parameters), evaluating force and stiffness integrals across every element leads to an online computational bottleneck. The Energy-Conserving Sampling and Weighting (ECSW) method overcomes this issue by selecting a sparse subset of active elements E^* to evaluate reduced internal operators without full-mesh assembly.

[1] A application is a real-time virtual representation of a physical system that mirrors its operational states and dynamics through fast, data-informed numerical models [10, 14].

Combining these three techniques allows real-time evaluation of a cantilever beam with a geometrical defect across varying parameters (x_c, r) by performing all computations on a fixed reference domain $\hat{\Omega}$.

1.3 Research Objectives

The primary objective of this internship project is to establish an accessible, computationally efficient framework for real-time parameterized structural dynamics. Beyond mathematical rigor, the core ambition is to create an engaging pedagogical tool for classrooms, enabling teachers and students to interactively explore and visualize transient dynamic simulations with instantaneous feedback. Furthermore, the modular architecture is designed as an extensible foundation for future engineering applications (such as structural health monitoring and real-time control).

The specific contributions of this thesis are:

1. **Parameterized Reference Formulation**: Formulate the continuous pullback mapping $\hat{\boldsymbol{\Psi}}$ to enable constant mesh topology and vector-space compatibility across varying geometric parameters $\boldsymbol{\mu} = (x_c, r)^T$.

2. **Snapshot Collection & Modal Compression**: Perform transient dynamic sweeps of the full-order IGA system and construct an optimal POD reduced basis capturing the dominant structural dynamics.

3. **Sparse Hyper-Reduction Optimization**: Calibrate an Energy-Conserving Sampling and Weighting (ECSW) scheme to select a minimal element subset E^*, achieving strict sub-millisecond assembly times.

4. **Interactive Pedagogical Deployment**: Implement the complete offline/online reduced solver within a standalone, client-side WebGL application (written natively in JavaScript and executed on a single processor core). This provides a zero-install, hands-on learning environment for students and teachers (`https://cnam-internship-mumen.netlify.app/`) and lays the technical groundwork for future web-based interactive web applications.

1.4 State of the Art

1.4.1 Isogeometric Analysis (IGA)

Introduced by Hughes et al. in 2005 [18], Isogeometric Analysis (IGA) closes the gap between Computer-Aided Design (CAD) and Finite Element Analysis (FEA). By utilizing NURBS basis functions for the analysis, IGA offers several key advantages:

- Exact geometric representation of curved boundaries without discretization errors.

- Higher-order continuity (C^{p-1} for degree p), which leads to superior accuracy per degree of freedom compared to classical C^0 Lagrange finite elements.

- Native support for mesh refinement techniques (knot insertion, degree elevation, and k-refinement) while preserving the exact CAD geometry.

1.4.2 Reduced Order Modeling (ROM) for Parameterized Systems

Projection-based Reduced Order Modeling enables a compression of these high-dimensional systems by approximating the high-fidelity Full Order Model (FOM) solution in a low-dimensional subspace. This framework operates in an intrusive context where we have full access to the solver. The methodology is split into two phases:

1. **Off-line Phase**: This involves data generation, basis generation, and model reduction by projection (pre-computing the necessary operators).

2. **On-line Phase**: This enables the fastest computation of a quantity of interest after reduction, which can be done by resolution of the reduced solution or interpolation of reduced coordinates.

Standard basis generation techniques include Proper Orthogonal Decomposition (POD) [3], Proper Generalized Decomposition (PGD) [10], Reduced Basis (RB) methods [26], and Krylov subspace methods [27]. For projection-based ROM, the framework by Willcox [29] provides fundamental insights. In a parameterized context, the reduced basis $\mathbf{\Phi}$ must capture the system's behavior across the entire parameter domain.

However, geometrical parameters are most of the time non-affine, meaning there is no separation of operators and parameters. This context corresponds to the "bottleneck of ROM" because high-dimensional matrices and vectors have to be computed online despite the existence of the reduction basis.

1.4.3 Shape Parameterization & Reference Configurations

To resolve the vector space incompatibility and avoid rebuilding the mesh, the governing equations are pulled back from the parameterized physical domain $\Omega(\boldsymbol{\mu})$ to a fixed, parameter-independent reference configuration $\hat{\Omega}$. In the literature, geometrical parameterization is often handled using Free-Form Deformation (FFD) and radial basis functions [26, 1], or through Arbitrary Lagrangian-Eulerian (ALE) formulations common in Fluid-Structure Interaction (FSI). In this work, the use of IGA allows explicit parameterization of the geometry directly through the positions and weights of the Control Points (for example, one can explicitly set a radius parameter for a circular hole). By applying a smooth geometric mapping $\mathbf{\Psi}(\cdot; \boldsymbol{\mu})$, the integrals are evaluated on $\hat{\Omega}$. Under IGA, the parent parametric domain $\hat{\Omega}_{\text{param}} = [0, 1]^2$ serves natively as this reference configuration, providing a mathematically elegant and structurally invariant framework for the assembly process.

1.4.4 Hyper-Reduction Methods

Even when the equations are defined on the reference configuration, geometrical parameters introduce a computational bottleneck because the full-order integrals depend on parameter-dependent Jacobians. To overcome this bottleneck, hyper-reduction methods are employed. Existing hyper-reduction techniques include the Discrete Empirical Interpolation Method (DEIM / MDEIM) [9, 5], ECSW, and POD-GNAT.

While point-collocation techniques like DEIM are easy to implement, they do not conserve energy and can violate the symmetric positive-definite (SPD) structure of elastic stiffness matrices. In contrast, the Energy-Conserving Sampling and Weighting (ECSW) method [14, 13] replaces global domain integration with a sparse, non-negatively weighted sum over a minimal active element subset $\Omega_{\text{ECSW}} \subset \hat{\Omega}$. ECSW is perfect for dynamic simulations with a specified tolerance because it strictly preserves the symmetry, coercivity, and energy conservation properties of the governing continuum equations.

1.5 Thesis Roadmap & Chapter Structure

The manuscript is structured into the logical progression illustrated in Figure 1.1:

Chapter 2: Theory & Problem Formulation
Continuous governing equations for 2D linear plane stress structural dynamics, parameterized beam geometry, coordinate mappings.

↓

Chapter 3: Discretization via Isogeometric Analysis
B-Spline and NURBS definitions, knot vectors, patches, control points and weights, convergence analysis.

↓

Chapter 4: Pullback Mapping & Reference Domain Integration
Formulation written on a reference configuration $\hat{\Omega}$ (fictitious displacement, transformation Jacobian). Comparison between problem on reference domain and parameterized domain.

↓

Chapter 5: Full Order Model (FOM) Simulation
Generalized-α time integration, snapshot generation, convergence analysis.

↓

Chapter 6: Reduced Order Modeling
Off-line phase (hyper-reduction, POD basis generation, ECSW weights pre-computation). On-line phase on an interactive webpage (WebGL architecture[2]).

↓

Chapter 7: Conclusion & Future Perspectives
Outcomes, performance synthesis, primary research milestones, future optimization vectors.

Figure 1.1: Inter-chapter dependency and thematic workflow roadmap of the thesis.

This inter-chapter workflow (Figure 1.1) connects the continuous continuum mechanics formulation directly to the real-time WebGL application execution.

[2]WebGL (Web Graphics Library) is a JavaScript API for rendering interactive 2D and 3D graphics within any compatible web browser without the use of plug-ins [23].

1.6 Chapter Summary

Chapter Summary: Objectives & Methodological Synergy

Challenge	Resolving transient structural dynamics in real time ($\geq$ 60 FPS) under changing geometry parameters without computationally expensive mesh rebuilding.
Core Idea	Combine **Isogeometric Analysis (IGA)** for exact CAD geometry representation with **Reduced Order Modeling (ROM)** and **ECSW Hyper-Reduction** on a fixed reference configuration.
Key Targets	Sub-millisecond online solve time (< 1.5 ms), high physical fidelity (L_2 error $< 0.1\%$), and native WebGL application integration.

Chapter 2

Theory: Parameterized Continuum Mechanics

Developing reduced order models for parameterized structures requires a firm foundation in continuum elastodynamics and formal shape parameterization. This chapter establishes the strong and weak variational formulations of linear elasticity, introduces the parameter space $\boldsymbol{\mu} = (x_c, r)^T$ on an illustrative cantilever beam example with a geometrical defect used throughout the thesis (with straightforward extension to general parameterizations), and details the mathematical challenges introduced by domain variations.

2.1 Geometry Configurations & Shape Parameterization

We begin by defining the parameterized domain geometry $\Omega(\boldsymbol{\mu})$ and establishing how shape variations alter the physical configuration of the structural beam.

Let $\Omega(\boldsymbol{\mu}) \subset \mathbb{R}^2$ define the parameterized physical configuration of a structural continuum. As illustrated in Figure 2.1, we consider a 2D plane stress problem referred to as the notched cantilever beam, featuring symmetric Gaussian notches along its upper and lower boundaries. This serves as an application example written throughout the thesis using two parameters (of course, an extension to a general case is straightforward).

The shape variation is controlled by a parameter vector $\boldsymbol{\mu} = (x_c, r)^T \in \mathcal{D} \subset \mathbb{R}^2$:

- **Notch Position** (x_c): The longitudinal position of the notch center from the clamped left support ($2.5\,\mathrm{m} \leq x_c \leq 7.5\,\mathrm{m}$ for beam length $L = 8.0\,\mathrm{m}$).
- **Notch Depth** (r): The depth parameter controlling the symmetric Gaussian boundary cutouts ($0.0\,\mathrm{m} \leq r \leq 0.75\,\mathrm{m}$, where $r = 0$ corresponds to the pristine straight beam).

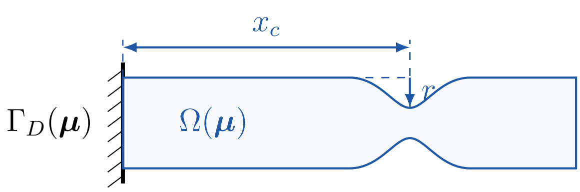


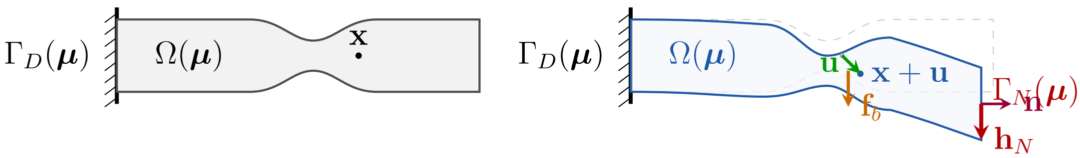
Figure 2.1: Schematic of the parameterized notched cantilever beam configuration $\Omega(\boldsymbol{\mu})$ showing Dirichlet boundary support $\Gamma_D(\boldsymbol{\mu})$, notch location x_c, and radius r.

The structural parameterization illustrated above (Figure 2.1) defines the design space $\mathcal{D}$ via notch position x_c and cutout radius r.

2.2 Governing Elastodynamic Equations & Kinematics

Next, we formulate the strong form balance of linear momentum and Cauchy stress-strain relationships under small-strain elastodynamics.

Kinematic deformation from the initial physical configuration $\Omega(\boldsymbol{\mu})$ to the deformed state under external body forces $\mathbf{f}_b$ and boundary tractions $\mathbf{h}_N$ is illustrated below (Figure 2.2). It is crucial to note that because the problem is linear, the "deformed geometry" is confounded with the "initial state". Furthermore, to avoid any misunderstanding with nonlinear mechanics terminology (where one distinguishes between a "reference configuration" and a "current configuration"), we explicitly differentiate between the *reference parameter configuration* $\hat{\Omega}$ (introduced in Chapter 4) and the *parameterized physical configuration* $\Omega(\boldsymbol{\mu})$, deliberately avoiding the term "current" throughout the text.



(a) Undeformed Configuration **(b) Deformed Configuration**

Figure 2.2: Kinematic mapping and boundary conditions of the parameterized notched cantilever beam.

Comparing the undeformed reference geometry $\Omega(\boldsymbol{\mu})$ to the deformed state under applied loads $\mathbf{f}_b$ and $\mathbf{h}_N$ (Figure 2.2) demonstrates the displacement field vector $\mathbf{u}(t)$.

Table 2.1: Definition of kinematic notation and continuum domain boundaries.

Symbol	Physical Meaning / Definition
$\Omega(\boldsymbol{\mu})$	Parameter-dependent physical domain configuration
$\Gamma_D(\boldsymbol{\mu})$	Fixed Dirichlet boundary at left clamped support ($\mathbf{u} = \mathbf{0}$)
$\Gamma_N(\boldsymbol{\mu})$	Neumann traction boundary subjected to dynamic loading
$\mathbf{u}(\mathbf{x}, t; \boldsymbol{\mu})$	Spatial displacement field vector at position $\mathbf{x}$ and time t
$\boldsymbol{\sigma}(\mathbf{u})$	Cauchy stress tensor under linear elasticity
$\boldsymbol{\varepsilon}(\mathbf{u})$	Small-strain tensor: $\boldsymbol{\varepsilon}(\mathbf{u}) = \frac{1}{2}\left(\nabla\mathbf{u} + (\nabla\mathbf{u})^T\right)$
$\mathbf{D}$	Isotropic constitutive elasticity matrix in Voigt notation

This formal notation (Table 2.1) establishes the continuum framework used throughout the initial configuration and small-strain kinematic relations. For the sake of readability, the parametric and temporal dependency of the displacement field $\mathbf{u}(\mathbf{x}, t; \boldsymbol{\mu})$ will often be abbreviated simply as $\mathbf{u}$.

Assuming small strains and linear isotropic elasticity, the problem considered consists in finding the displacement field $\mathbf{u}$ such that the strong form governing linear momentum balance on $\Omega(\boldsymbol{\mu}) \times (0, T]$ is given as:

$$\underbrace{\rho\ddot{\mathbf{u}}}_{\text{Inertial Force}} - \underbrace{\nabla\cdot\boldsymbol{\sigma}(\mathbf{u})}_{\text{Internal Stress Divergence}} = \underbrace{\mathbf{f}_b}_{\text{Body Force}} \tag{2.1}$$

subject to boundary conditions $\mathbf{u} = \mathbf{0}$ on $\Gamma_D(\boldsymbol{\mu})$, traction $\boldsymbol{\sigma} \cdot \mathbf{n} = \mathbf{h}_N$ on $\Gamma_N(\boldsymbol{\mu})$, and initial conditions $\mathbf{u}(\mathbf{x}, 0) = \mathbf{u}_0(\mathbf{x})$, $\dot{\mathbf{u}}(\mathbf{x}, 0) = \mathbf{v}_0(\mathbf{x})$.

2.3 Weak Variational Formulation

We now derive the weak variational form of elastodynamics, establishing the kinetic energy, strain energy, and external work forms required for numerical discretization.

Multiplying by kinematically admissible test functions $\delta\mathbf{u} \in \mathcal{V}_0(\Omega(\boldsymbol{\mu})) = \{\mathbf{v} \in [H^1(\Omega(\boldsymbol{\mu}))]^2 \mid \mathbf{v} = \mathbf{0} \text{ on } \Gamma_D(\boldsymbol{\mu})\}$ and applying integration by parts and Green's formulas yields the weak form:

$$\underbrace{m(\ddot{\mathbf{u}}, \delta\mathbf{u}; \boldsymbol{\mu})}_{\text{Virtual Kinetic Energy}} + \underbrace{k(\mathbf{u}, \delta\mathbf{u}; \boldsymbol{\mu})}_{\text{Virtual Internal Energy}} = \underbrace{f(\delta\mathbf{u})}_{\text{Virtual External Energy}} \qquad \forall \delta\mathbf{u} \in \mathcal{V}_0(\Omega(\boldsymbol{\mu})) \tag{2.2}$$

where the bilinear forms and load linear form are given as:

$$m(\ddot{\mathbf{u}}, \delta\mathbf{u}; \boldsymbol{\mu}) = \int_{\Omega(\boldsymbol{\mu})} \rho \ddot{\mathbf{u}} \cdot \delta\mathbf{u} \, d\Omega \tag{2.3}$$

$$k(\mathbf{u}, \delta\mathbf{u}; \boldsymbol{\mu}) = \int_{\Omega(\boldsymbol{\mu})} \boldsymbol{\varepsilon}(\delta\mathbf{u})^T \mathbf{D} \boldsymbol{\varepsilon}(\mathbf{u}) \, d\Omega \tag{2.4}$$

$$f(\delta\mathbf{u}) = \int_{\Omega(\boldsymbol{\mu})} \mathbf{f}_b \cdot \delta\mathbf{u} \, d\Omega + \int_{\Gamma_N(\boldsymbol{\mu})} \mathbf{h}_N \cdot \delta\mathbf{u} \, d\Gamma \tag{2.5}$$

Note that the geometric parameter $\boldsymbol{\mu}$ explicitly appears in the domain of integration $\Omega(\boldsymbol{\mu})$ for each of these virtual energy terms.

2.4 The Shape Parameterization Challenge

The following analysis highlights why physical domain variations break conventional snapshot POD extraction, motivating the transition to reference domain pullback integration.

Evaluating $k(\mathbf{u}, \delta\mathbf{u}; \boldsymbol{\mu})$ on a varying domain $\Omega(\boldsymbol{\mu})$ introduces two critical computational hurdles at the discrete level: physical remeshing destroys mathematical consistency for the ROM using POD basis, and online assembly over the full physical mesh causes severe latency.

To address these hurdles, we compare naive physical remeshing against reference pullback mapping across three key criteria:

- **Mathematical Consistency**: Physical remeshing alters node counts and ordering across parameters, ruining mathematical consistency for the ROM using POD basis because vectors of different dimensions cannot be combined into a single snapshot matrix. In contrast, reference pullback mapping evaluates all snapshots on an invariant domain $\hat{\Omega}$, guaranteeing SVD vector consistency.

- **Online Assembly Speed**: Physical remeshing requires rebuilding and re-integrating the entire mesh at every parameter update. Pullback mapping allows precomputing reference terms, reducing online assembly to a minimal active element subset E^*.

To guarantee mathematical consistency and achieve sub-millisecond solves, this thesis selects the **Diffeomorphic Pullback Mapping** approach, shifting parameter dependencies entirely into integration Jacobians on a fixed reference space $\hat{\Omega}$ (detailed in Chapter 4).

2.5 Chapter Summary

In summary, we synthesize the continuum elastodynamic foundation established in this chapter and preview the spatial discretization techniques introduced in Chapter 3.

Chapter Summary: Continuum Mechanics & Shape Parameterization

Physical Domain A 2D plane stress problem referred to as the notched cantilever beam, with parameterized configuration $\Omega(\boldsymbol{\mu}) \subset \mathbb{R}^2$ and design parameters $\boldsymbol{\mu} = (x_c, r)^T$ (notch location, standard deviation).

Governing Law Linear elastodynamics balance of momentum $\rho\ddot{\mathbf{u}} - \nabla \cdot \boldsymbol{\sigma} = \mathbf{f}_b$ subject to Dirichlet clamping and Neumann tractions.

Core Bottleneck Domain variation $\Omega(\boldsymbol{\mu})$ destroys vector space compatibility across parameter instances, preventing direct snapshot POD extraction.

Key Takeaway: We established the elastodynamic continuum formulation for the notched cantilever beam and demonstrated that direct physical evaluation on $\Omega(\boldsymbol{\mu})$ breaks state vector compatibility across parameters. In Chapter 3, we discretize this formulation using Isogeometric Analysis (IGA) prior to pulling back integration terms to a reference domain in Chapter 4.

Chapter 3

Discretization: Isogeometric Analysis (IGA)

Following the continuum elastodynamic formulation established in Chapter 2, discretizing the governing equations requires a spatial basis capable of handling complex geometry accurately. Traditional finite element methods introduce geometric approximation errors when tessellating curved boundaries into polygonal meshes because the geometry depends on the field discretization using Lagrange polynomials. Isoparametric FEM implies that the geometry shares the same parameterization as the field discretization, whereas Isogeometric Analysis (IGA) dictates that the field shares the same parameterization as the exact geometry construction.

It is important to note that while IGA utilizes CAD basis functions (NURBS), directly extracting full 3D solid tensor-product meshes from standard CAD files remains an open challenge, as most CAD formats solely provide boundary-representation (B-Rep) shells. Nevertheless, for the 2D geometries considered in this thesis, IGA successfully employs NURBS as basis functions for both exact boundary representation and elastodynamic state approximation, yielding continuous semi-discrete equations of motion.

3.1 B-Splines & NURBS Geometry Representation

Here, we introduce univariate B-splines, rational NURBS basis functions, and tensor-product surface patches that form the foundation of IGA.

Isogeometric Analysis (IGA), pioneered by Hughes et al. [18] and expanded by Cottrell et al. [11], establishes a link between Computer-Aided Design (CAD) and Finite Element Analysis (FEA) by employing NURBS basis functions directly in the analysis solver.

3.1.1 Univariate B-Spline Basis & Cox-de Boor Recurrence

The Cox-de Boor algorithm is fundamentally ubiquitous in B-spline evaluations [24] due to its numerical stability and efficient recursive formulation. Cox-de Boor recurrence relations and derivative formulas for univariate B-spline basis functions are defined as follows.

Given an open knot vector $\Xi = \{\xi_0, \xi_1, \ldots, \xi_m\}$ (where $m+1$ is the total number of knots), univariate B-spline basis functions $N_{i,p}(\xi)$ of degree p are defined recursively. A critical feature of open knot vectors is knot repetition: repeating the first and last knots $p+1$ times ensures the basis functions interpolate the endpoints, which is crucial for applying boundary conditions.

$$N_{i,0}(\xi) = \begin{cases} 1 & \text{if } \xi_i \le \xi < \xi_{i+1} \\ 0 & \text{otherwise} \end{cases} \tag{3.1}$$

$$N_{i,p}(\xi) = \frac{\xi - \xi_i}{\xi_{i+p} - \xi_i} N_{i,p-1}(\xi) + \frac{\xi_{i+p+1} - \xi}{\xi_{i+p+1} - \xi_{i+1}} N_{i+1,p-1}(\xi) \tag{3.2}$$

Analytical spatial gradients for strain evaluations are computed using derivative recurrence:

$$\frac{d}{d\xi} N_{i,p}(\xi) = p \left(\frac{N_{i,p-1}(\xi)}{\xi_{i+p} - \xi_i} - \frac{N_{i+1,p-1}(\xi)}{\xi_{i+p+1} - \xi_{i+1}} \right) \tag{3.3}$$

Evaluating quadratic B-spline basis functions $N_{i,2}(\xi)$ over an open knot vector yields smooth localized shape functions with partition of unity properties. In Figure 3.1, one can see how these quadratic basis functions distribute across the parametric space ξ.

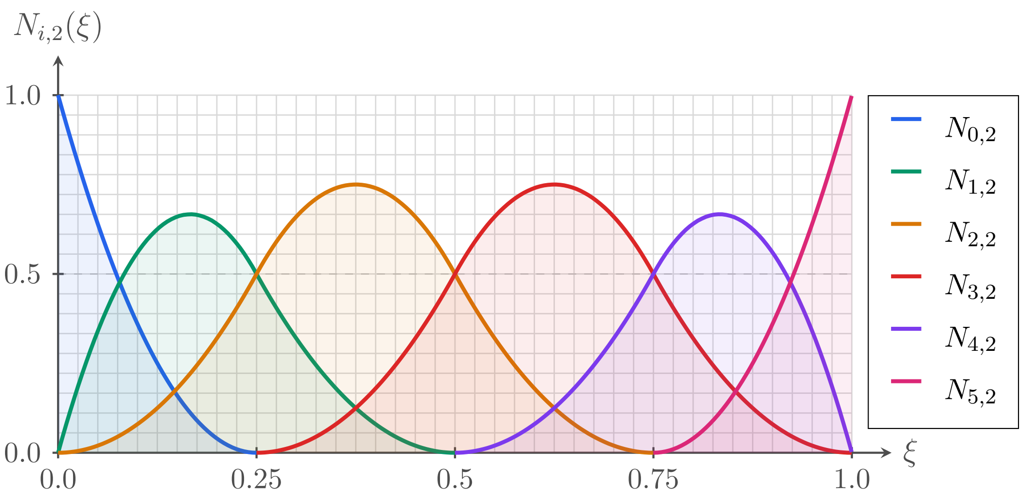


Figure 3.1: Quadratic B-spline basis functions $N_{i,2}(\xi)$ evaluated over knot vector $\Xi = \{0, 0, 0, 0.25, 0.5, 0.75, 1, 1, 1\}$.

3.1.2 Non-Uniform Rational B-Splines (NURBS)

B-spline basis functions are fundamental ingredients to construct geometries (curves, surfaces, and volumes). Before going further, one must introduce the concept of NURBS basis functions first. Standard B-splines are not able to represent conic sections (such as circles and ellipses) exactly. Because these shapes are extremely common in engineering, rational basis functions with projective weights are formulated to represent exact conic and curved geometries. For example, the exact circular geometry shown later in Figure 3.7 can only be constructed using NURBS, clearly highlighting the difference from standard B-spline curves.

Projective control weights w_i define the rational basis functions $R_{i,p}(\xi)$:

$$R_{i,p}(\xi) = \frac{N_{i,p}(\xi) w_i}{\sum_{j=0}^{n-1} N_{j,p}(\xi) w_j} = \frac{N_{i,p}(\xi) w_i}{W(\xi)} \tag{3.4}$$

Spatial gradients of rational basis functions are evaluated using the quotient rule:

$$\frac{d}{d\xi} R_{i,p}(\xi) = w_i \frac{\frac{dN_{i,p}(\xi)}{d\xi} W(\xi) - N_{i,p}(\xi) \frac{dW(\xi)}{d\xi}}{[W(\xi)]^2} \tag{3.5}$$

Spatial coordinates of a 1D parametric curve $\mathbf{C}(\xi)$ are constructed as a linear combination of control points $\mathbf{P}_i$ weighted by rational basis functions $R_{i,p}(\xi)$:

$$\mathbf{C}(\xi) = \sum_{i=0}^{n-1} R_{i,p}(\xi)\mathbf{P}_i \tag{3.6}$$

Figure 3.2 illustrates the geometric construction of a 1D B-spline curve $\mathbf{C}(\xi)$ blending a control polygon formed by control points $\mathbf{P}_0$ through $\mathbf{P}_4$.

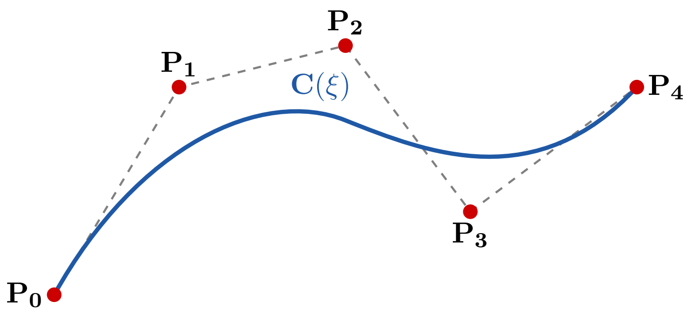


Figure 3.2: Geometric construction of a 1D B-spline curve $\mathbf{C}(\xi)$ blended by control points $\mathbf{P}_i$ and control polygon (dashed lines).

Evaluating these quadratic basis functions over active knot spans (Figure 3.1) confirms their local support and partition of unity properties.

3.1.3 Bivariate Tensor-Product NURBS Surface Patches

To construct a surface, one performs a tensor product of two 1D B-spline basis function sets across knot vectors Ξ and $\mathcal{H}$. Weight factors $w_{i,j}$ define rational bivariate basis functions $R_{i,j}^{p,q}(\xi,\eta)$:

$$R_{i,j}^{p,q}(\xi,\eta) = \frac{N_{i,p}(\xi)M_{j,q}(\eta)w_{i,j}}{\sum_{k=0}^{n-1}\sum_{l=0}^{m-1} N_{k,p}(\xi)M_{l,q}(\eta)w_{k,l}} \tag{3.7}$$

Evaluating bivariate tensor-product NURBS basis functions yields 2D shape functions over the parametric domain (Figure 3.3).

The physical domain surface, where a physical point inside the surface is denoted by $\mathbf{x}(\xi,\eta)$, is constructed as a linear combination of a 2D control net $\mathbf{P}_{i,j}$, as shown in Figure 3.4:

$$\mathbf{x}(\xi,\eta) = \sum_{i=0}^{n-1}\sum_{j=0}^{m-1} R_{i,j}^{p,q}(\xi,\eta)\mathbf{P}_{i,j} \tag{3.8}$$

For surface integration, one can compute the 2D coordinate Jacobian matrix $\mathbf{J}(\xi,\eta)$ as:

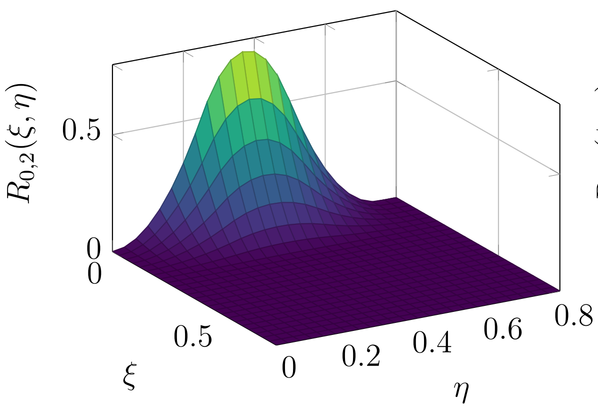
$$\mathbf{J}(\xi,\eta) = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} \end{bmatrix} = \begin{bmatrix} \sum \frac{\partial R_{i,j}^{p,q}}{\partial \xi}(\mathbf{P}_{i,j})_x & \sum \frac{\partial R_{i,j}^{p,q}}{\partial \xi}(\mathbf{P}_{i,j})_y \\ \sum \frac{\partial R_{i,j}^{p,q}}{\partial \eta}(\mathbf{P}_{i,j})_x & \sum \frac{\partial R_{i,j}^{p,q}}{\partial \eta}(\mathbf{P}_{i,j})_y \end{bmatrix} \tag{3.9}$$

The local outward unit surface normal vector $\mathbf{n}(\xi,\eta)$ is extracted via the cross product:

$$\mathbf{n}(\xi,\eta) = \frac{\mathbf{g}_\xi \times \mathbf{g}_\eta}{\|\mathbf{g}_\xi \times \mathbf{g}_\eta\|} \tag{3.10}$$

where $\mathbf{g}_\xi = \partial\mathbf{S}/\partial\xi$ and $\mathbf{g}_\eta = \partial\mathbf{S}/\partial\eta$.

(a) Boundary basis function $R_{0,2}(\xi, \eta)$ (b) Internal basis function $R_{2,2}(\xi, \eta)$



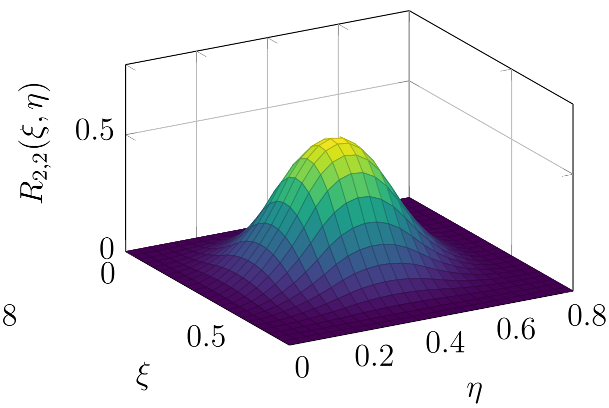


Figure 3.3: 3D surface plots of two distinct bivariate tensor-product B-spline basis functions: (a) boundary basis function $R_{0,2}(\xi, \eta) = N_{0,2}(\xi)M_{2,2}(\eta)$ and (b) internal basis function $R_{2,2}(\xi, \eta) = N_{2,2}(\xi)M_{2,2}(\eta)$, demonstrating edge-supported versus fully internal support.

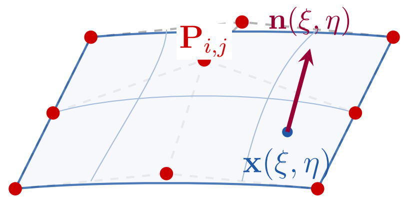


Figure 3.4: Bivariate B-spline surface patch $\mathbf{x}(\xi, \eta)$ guided by a control net $\mathbf{P}_{i,j}$, with local surface normal $\mathbf{n}(\xi, \eta)$.

By linking parametric coordinates to a 3D control net $\mathbf{P}_{i,j}$ (Figure 3.4), bivariate NURBS patches construct exact geometric surfaces.

3.2 Refinement Strategies: h-, p-, & k-Refinement

A reduced number of control points enables us to construct the exact curved geometry. Nevertheless, the field discretization accuracy depends heavily on the number of control points. When there are significant fluctuations of the elastodynamic field inside the domain, one must introduce the concept of mesh refinement in IGA. By mesh refinement, it means we can add additional control points without compromising the exact physical domain.

The following discussion details the three refinement mechanisms of IGA that enable mesh enrichment without altering the exact CAD geometry.

A major advantage of IGA is mesh refinement without modifying CAD geometry definitions:

Comparing these three refinement pathways (Table 3.1) shows that IGA enables arbitrary mesh enrichment while maintaining strict C^{p-1} continuity and exact CAD boundary definitions.

Table 3.1: Comparative synthesis of IGA refinement mechanisms.

Refinement Type	Mechanism	Continuity	Exact CAD Geometry	Primary Advantage / Impact
h-Refinement	Knot Insertion	Retained (C^{p-1})	Preserved Exactly	Global mesh enrichment (1D local knot insertion propagates entirely across 2D patches via tensor product).
p-Refinement	Degree Elevation	Increased (C^p)	Preserved Exactly	Higher-order accuracy without Runge oscillatory phenomena.
k-Refinement	Order Elevation	Maximum (C^{p-1})	Preserved Exactly	Superior accuracy per DOF; non-local basis with high continuity.

3.2.1 Boehm Knot Insertion Algorithm (h-Refinement)

Boehm's algorithm [24] for knot insertion (h-refinement) enables local mesh refinement near stress concentrations.

In h-refinement, new control points $\mathbf{P}_i^{\text{new}}$ are computed via Boehm's algorithm:

$$\mathbf{P}_i^{\text{new}} = \alpha_i \mathbf{P}_i + (1 - \alpha_i)\mathbf{P}_{i-1} \tag{3.11}$$

where blending coefficients α_i are:

$$\alpha_i = \begin{cases} 1 & \text{if } i \leq k - p \\ \frac{\bar{\xi} - \xi_i}{\xi_{i+p} - \xi_i} & \text{if } k - p + 1 \leq i \leq k \\ 0 & \text{if } i \geq k + 1 \end{cases} \tag{3.12}$$

For 2D domains with lattice matrix $\mathbf{P}$, directional transformations $\mathbf{T}_\xi$ and $\mathbf{T}_\eta$ yield:

$$\mathbf{P}^{\text{new}} = \mathbf{T}_\xi \mathbf{P} \mathbf{T}_\eta^T, \quad \mathbf{P}^{\text{new}} = (\mathbf{T}_\eta \otimes \mathbf{T}_\xi)\,\mathbf{P} \tag{3.13}$$

Degree elevation (p-refinement) operates via $\mathbf{P}^{\text{new}} = \mathbf{T}_p \mathbf{P}$.

Demonstrating h-refinement (knot insertion) confirms how the control polygon refines while preserving exact curve geometry (Figure 3.5).

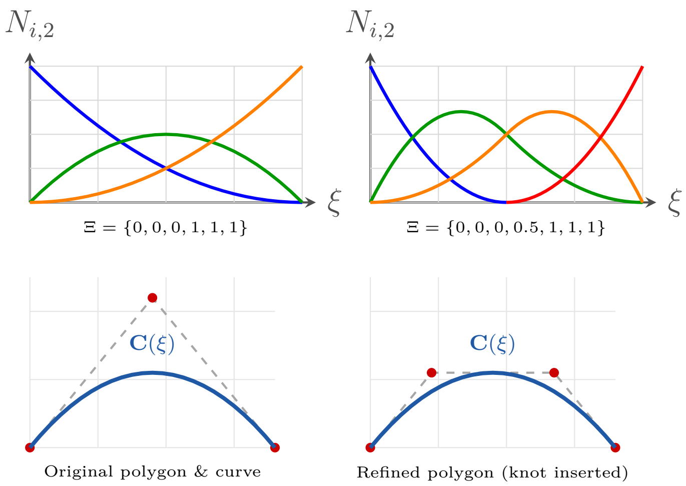


Figure 3.5: h-refinement (knot insertion) schema showing basis functions before/after inserting knot $\xi = 0.5$ (top) and corresponding control polygon refinement while preserving exact curve geometry (bottom).

Inserting knots via Boehm's algorithm (Figure 3.5) enriches the control polygon density while preserving the exact CAD geometry.

3.2.2 Degree Elevation (p-Refinement)

Degree elevation (p-refinement) increases the polynomial order p of the NURBS basis functions without modifying the underlying CAD geometry or introducing new knot spans.

Starting from a curve of polynomial degree p defined over knot vector $\Xi = \{\xi_0, \xi_1, \ldots, \xi_m\}$, degree elevation increases the polynomial order from p to $p+q$ (where q is the elevation increment). To preserve the original continuity C^{p-m_i} at each distinct knot ξ_i with original multiplicity m_i, the multiplicity of each knot in the elevated knot vector Ξ^{new} is increased to $m_i + q$.

The updated control points $\mathbf{P}_i^{\text{new}}$ are computed via a degree elevation transformation matrix $\mathbf{T}_p$:

$$\mathbf{P}^{\text{new}} = \mathbf{T}_p \mathbf{P} \tag{3.14}$$

Unlike classical finite element p-refinement, IGA degree elevation preserves exact geometry and continuity C^{p-m_i} across knot spans while completely eliminating Runge oscillatory phenomena (Figure 3.6).

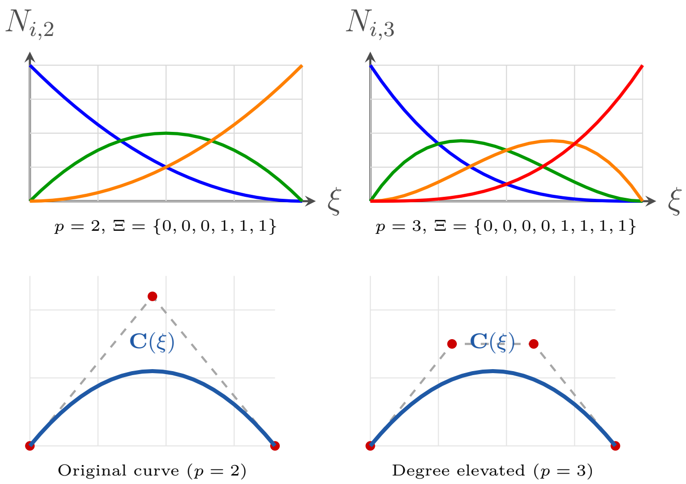


Figure 3.6: p-refinement (degree elevation) schema showing basis functions (top) and B-spline curves (bottom) before/after elevating polynomial degree from $p = 2$ to $p = 3$.

3.2.3 Order Elevation (k-Refinement)

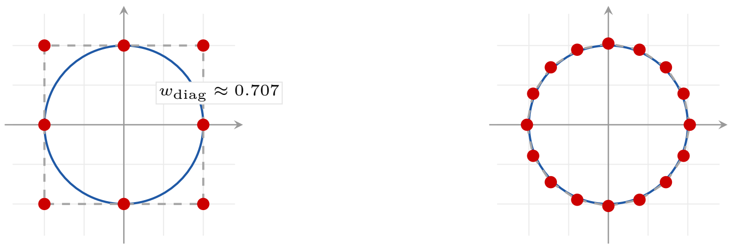
Unique to Isogeometric Analysis, k-refinement commutes degree elevation and knot insertion to construct high-order basis functions with maximum inter-element continuity (C^{p-1}).

In traditional finite element analysis (or standard p-refinement applied after mesh subdivision), knot insertion followed by degree elevation results in C^0 continuity across element boundaries. In contrast, k-refinement executes these operations in reverse order:

1. **Degree Elevation First**: The polynomial degree of the coarse curve or surface patch is elevated from p to $p+q$, raising internal continuity to C^{p+q-1}.

2. **Knot Insertion Second**: Knots are subsequently inserted into the elevated knot vector, retaining the maximum achievable inter-element continuity C^{p+q-1} across all newly created knot spans.

By maintaining maximum inter-element continuity, k-refinement significantly reduces the total number of active control variables (degrees of freedom) required per element compared to standard p-FEM, offering superior spectral accuracy per degree of freedom for dynamic structural simulations. These powerful properties are formulated directly inside the patch and only exist for k-refinement. That is why k-refinement is always preferable for high accuracy simulations (and notably, it does not exist in classical FEM).



(a) Initial Geometry (8 Control Points) **(b) After k-Refinement (16 Control Points)**

Figure 3.7: k-refinement (order elevation) schema demonstrating exact circle CAD geometry preservation with increased control point density and maximum inter-element continuity C^{p-1}.

3.3 Isogeometric System Matrix Assembly

Here, we detail the Galerkin projection of elastodynamic fields onto NURBS basis functions to assemble global mass $\mathbf{M}$, stiffness $\mathbf{K}$, and force $\mathbf{F}_{\text{ext}}$ matrices.

Approximating the displacement field using bivariate NURBS basis functions $R_a(\mathbf{x})$ yields:

$$\mathbf{u}_h(\mathbf{x}, t) = \sum_{a=1}^{n_b} R_a(\mathbf{x})\mathbf{d}_a(t) \tag{3.15}$$

where n_b is the total number of basis functions (degrees of freedom) after k-refinement. The explicit analytical derivatives $\frac{\partial R_a}{\partial \xi}$ and $\frac{\partial R_a}{\partial \eta}$ required for strain matrix assembly are detailed in Appendix 7.5, and $\mathbf{d}_a(t) \in \mathbb{R}^2$ represents control variable displacements. The test functions $\delta\mathbf{u}$ are represented using the same basis functions. Substituting the NURBS approximations for $\mathbf{u}_h$ and the virtual displacements (test functions) $\delta\mathbf{u}$ into the continuous weak form bilinear and linear operators:

$$m(\mathbf{u}, \mathbf{v}) = \int_\Omega \rho\mathbf{u} \cdot \mathbf{v}\, d\Omega \tag{3.16}$$

$$k(\mathbf{u}, \mathbf{v}) = \int_\Omega \varepsilon(\mathbf{u})^T \mathbf{D}\varepsilon(\mathbf{v})\, d\Omega \tag{3.17}$$

$$f(\mathbf{v}) = \int_\Omega \mathbf{f}_b \cdot \mathbf{v}\, d\Omega + \int_{\Gamma_N} \mathbf{h}_N \cdot \mathbf{v}\, d\Gamma \tag{3.18}$$

leads to the discrete system representation:

$$m(\ddot{\mathbf{u}}_h, \delta\mathbf{u}) = \sum_{a=1}^{n_b}\sum_{b=1}^{n_b} \delta\mathbf{d}_a^T \left(\int_\Omega \rho R_a R_b \mathbf{I}_2\, d\Omega\right) \ddot{\mathbf{d}}_b = \delta\mathbf{d}^T\mathbf{M}\ddot{\mathbf{d}} \tag{3.19}$$

$$k(\mathbf{u}_h, \delta\mathbf{u}) = \sum_{a=1}^{n_b}\sum_{b=1}^{n_b} \delta\mathbf{d}_a^T \left(\int_\Omega \mathbf{B}_a^T\mathbf{D}\mathbf{B}_b\, d\Omega\right) \mathbf{d}_b = \delta\mathbf{d}^T\mathbf{K}\mathbf{d} \tag{3.20}$$

$$f(\delta\mathbf{u}) = \sum_{a=1}^{n_b} \delta\mathbf{d}_a^T \left(\int_\Omega R_a\mathbf{f}_b\, d\Omega + \int_{\Gamma_N} R_a\mathbf{h}_N\, d\Gamma\right) = \delta\mathbf{d}^T\mathbf{F}_{\text{ext}} \tag{3.21}$$

Since the virtual control point displacements $\delta\mathbf{d}$ are arbitrary, this leads directly to the semi-discrete equations of motion (incorporating a proportional damping matrix $\mathbf{C}$):

$$\mathbf{M}\ddot{\mathbf{d}}(t) + \mathbf{C}\dot{\mathbf{d}}(t) + \mathbf{K}\mathbf{d}(t) = \mathbf{F}_{\text{ext}}(t) \tag{3.22}$$

Here, the global mass matrix $\mathbf{M}$, stiffness matrix $\mathbf{K}$, and external force vector $\mathbf{F}_{\text{ext}}$ are assembled from the local element integrals over knot spans Ω_e:

$$\mathbf{M}_{ab} = \int_{\Omega_e} \rho R_a R_b \mathbf{I}_2\, d\Omega \tag{3.23}$$

$$\mathbf{K}_{ab} = \int_{\Omega_e} \mathbf{B}_a^T\mathbf{D}\mathbf{B}_b\, d\Omega \tag{3.24}$$

$$\mathbf{F}_{\text{ext},a} = \int_{\Omega_e} R_a\mathbf{f}_b\, d\Omega + \int_{\Gamma_{N,e}} R_a\mathbf{h}_N\, d\Gamma \tag{3.25}$$

where $\mathbf{B}_a$ represents the strain-displacement matrix containing parametric derivatives of $R_a(\mathbf{x})$ mapped via the spatial Jacobian $\mathbf{J}(\xi, \eta)$.

To justify selecting the 80×40 element baseline for the full-order model, we perform a grid convergence study under static bending and validate accuracy against classical Lagrangian Finite Element methods.

3.4 Mesh Convergence & Accuracy Validation

Here, we state the physical load conditions, plane stress assumptions, and numerical setup governing the mesh convergence study, followed by accuracy benchmarks against Lagrangian FEM.

3.4.1 Simulation Conditions & Boundary Assumptions

The mesh convergence study models the notched cantilever beam from Figure 2.1 of length $L = 2.0$ m and height $H = 0.4$ m subjected to a static vertical tip load $F_y = -10$ kN applied at the free right boundary ($\xi = 1$). The fixed root boundary at $\xi = 0$ is fully clamped ($\mathbf{u} = \mathbf{0}$).

The material is assumed homogeneous, isotropic, and linearly elastic with Young's modulus $E = 210$ GPa, Poisson's ratio $\nu = 0.3$, and mass density $\rho = 7850$ kg/m^3. Under plane stress conditions, the constitutive tensor $\mathbf{D}$ is given by:

$$\mathbf{D} = \frac{E}{1-\nu^2}\begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix} \tag{3.26}$$

The strain-displacement matrix $\mathbf{B}_a$ for each control point a is constructed from the Cartesian derivatives of the NURBS basis functions ($\frac{\partial R_a}{\partial x}$, $\frac{\partial R_a}{\partial y}$), which are computed by mapping the parametric gradients ($\frac{\partial R_a}{\partial \xi}$, $\frac{\partial R_a}{\partial \eta}$) to the physical space using the inverse of the spatial Jacobian matrix $\mathbf{J}(\xi, \eta)$.

3.4.2 Grid Convergence Analysis via k-Refinement

To construct a convergent mesh while maximizing accuracy per degree of freedom, the convergence study investigates k-refinement across multiple polynomial orders ($p \in \{1, 2, 3\}$) and dyadic knot insertion levels (h-refinement from 10×5 up to 160×80 elements), as implemented in the custom JavaScript simulation engine (`iga-sim-5.3`[1]):

1. **Degree Elevation (p-Refinement)**: The initial CAD patch is elevated to linear ($p = 1, C^0$), quadratic ($p = 2, C^1$), and cubic ($p = 3, C^2$) representations.

2. **Knot Insertion (h-Refinement)**: Dyadic knot insertion is subsequently performed on each elevated patch, refining the mesh resolution from 10×5 (50 elements) up to a fine reference grid of 160×80 (12,800 elements).

Because degree elevation precedes knot insertion, all inserted internal knot spans preserve maximum C^{p-1} inter-element continuity. Vertical tip displacement U_y and relative error evaluated against the 160×80 fine reference solution ($U_y = 4.238$ mm) across degrees $p = 1, 2, 3$ are illustrated in Figure 3.8.

As evident in Figure 3.8, higher-order k-refinement ($p = 2, 3$) yields dramatically superior accuracy per degree of freedom compared to standard linear C^0 approximations. For quadratic k-refinement ($p = 2, C^1$), refining from 80×40 (3,200 elements, 6,888 DOFs) to 160×80 (12,800 elements, 26,568 DOFs) produces only a 1.24% shift in tip displacement while increasing equation count by 3.8×. This confirms 80×40 ($p = 2$) as the optimal, convergent baseline for full-order elastodynamic simulations.

[1]The open-source JavaScript implementation is available on GitHub, and the interactive simulation tool can be accessed online here

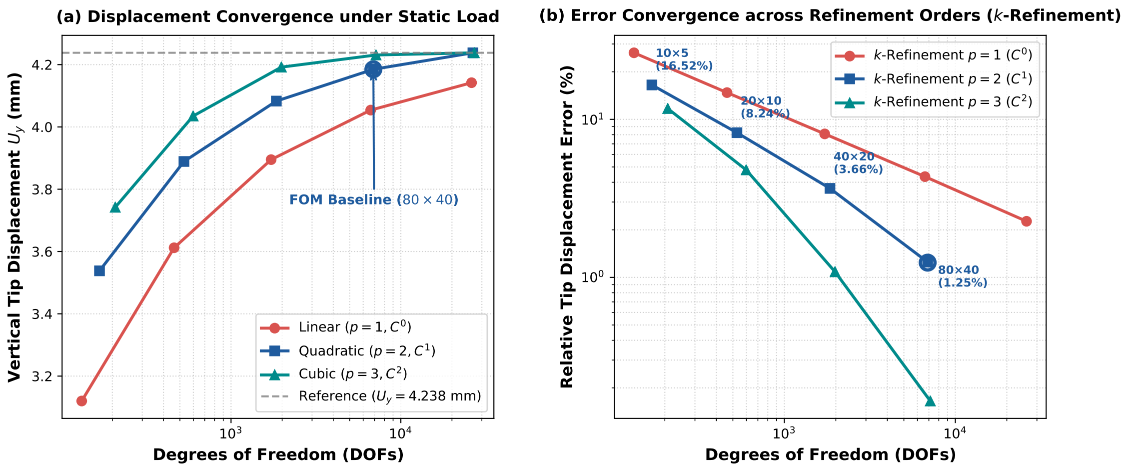


Figure 3.8: Multi-level k-refinement convergence study under static bending load: (a) vertical tip displacement U_y versus total Degrees of Freedom (DOFs) for linear ($p = 1$), quadratic ($p = 2$), and cubic ($p = 3$) NURBS discretizations; (b) relative tip displacement error (%) on log-log scale showing asymptotic convergence rates and justifying the 80×40 (6,888 DOFs, 1.24% relative error) FOM baseline.

3.4.3 Linear Accuracy Validation Against Lagrangian FEM

To assess the accuracy and computational efficiency of the bivariate NURBS discretization, the full-order IGA baseline (80×40 elements, quadratic C^1-continuous basis functions) is benchmarked against a fine-mesh classical Finite Element model utilizing C^0 Lagrangian linear elements (12,400 DOFs). The primary goal of this comparison is to highlight the advantages of Isogeometric Analysis over classical FEM: specifically, how the higher inter-element continuity of NURBS basis functions allows IGA to yield equivalent or superior accuracy in structural displacements and energy norms using a much smaller system size (fewer DOFs), which in turn significantly reduces the solver computation time. The comparative validation metrics are summarized in Table 3.2.

Table 3.2: Linear solution accuracy validation comparing full-order IGA degrees of freedom against fine-mesh Lagrangian FEM frameworks.

Mesh Setup	Total DoFs	Tip Disp. (mm)	Energy Norm (J)	Solve Time (ms)
Lagrangian FEM	12,400	4.182	1.042×10^5	845.2
IGA Core	**6,888**	**4.185**	**1.044×10^5**	**142.1**

Owing to the inter-element C^{p-1} continuity of quadratic NURBS basis functions, the IGA core achieves equivalent tip displacement accuracy (4.185 mm vs. 4.182 mm) with 44% fewer DOFs and a 6× reduction in linear solver computation time (142.1 ms vs. 845.2 ms).

3.5 Chapter Summary

In closing, we summarize the IGA discretization framework and establish the transition to Chapter 4 for reference domain pullback mapping.

Chapter Summary: CAD-Integrated Discretization

Core Paradigm	Use Non-Uniform Rational B-Splines (NURBS) as basis functions for both geometry definition and elastodynamic state approximation.
Refinement Vectors	h-refinement (knot insertion), p-refinement (degree elevation), and k-refinement (order elevation maintaining C^{p-1} continuity).
System Output	Semi-discrete matrix equations of motion $\mathbf{M}\ddot{\mathbf{d}} + \mathbf{C}\dot{\mathbf{d}} + \mathbf{K}\mathbf{d} = \mathbf{F}_{\text{ext}}$ without geometry discretization error.

Key Takeaway: Isogeometric Analysis discretizes the continuous elastodynamic equations using exact NURBS CAD geometry basis functions, assembling the full-order matrices $\mathbf{M}$, $\mathbf{K}$, and $\mathbf{C}$. In Chapter 4, we pull back these element integrals to a parameter-independent reference configuration $\hat{\Omega}$ to resolve shape parameter compatibility.

Chapter 4

Shape Parameterization: Reference Configuration Mapping

While Chapter 3 establishes exact NURBS-based spatial discretization on a single configuration, evaluating structural snapshots on varying physical meshes creates incompatible vector spaces across parameters, making global Model Order Reduction impossible. This chapter formulates the diffeomorphic pullback transformation $\boldsymbol{\Psi}(\hat{\mathbf{x}}; \boldsymbol{\mu})$, mapping all parameter-dependent physical configurations onto a single invariant reference domain $\hat{\Omega}$ to enable consistent POD basis extraction and hyper-reduction.

4.1 Coordinate Domain Mapping Architecture

We first establish the three-domain mapping framework linking parametric space $\hat{\Omega}_{\text{param}}$, reference domain $\hat{\Omega}$, and physical domain $\Omega(\boldsymbol{\mu})$.

To resolve vector space incompatibility across varying geometric parameters $\boldsymbol{\mu}$, we establish a three-domain mapping framework. As illustrated in Figure 4.1, the physical coordinates $\mathbf{x} \in \Omega(\boldsymbol{\mu})$ are expressed as a composition of reference coordinates $\hat{\mathbf{x}} \in \hat{\Omega}$ (which themselves are mapped from parametric space coordinates $\boldsymbol{\xi}$ via $\hat{\boldsymbol{\Psi}}(\boldsymbol{\xi})$) and a parameter-dependent geometric displacement field $\mathbf{d}_{\text{geo}}(\boldsymbol{\xi}, \boldsymbol{\mu})$. It is critically important to distinguish the geometric shape parameter $\boldsymbol{\mu}$ from the NURBS parametric coordinates $\boldsymbol{\xi}$, and to distinguish this geometric displacement $\mathbf{d}_{\text{geo}}$ from the physical structural displacement $\mathbf{u}$:

$$\mathbf{x} = \hat{\mathbf{x}} + \mathbf{d}_{\text{geo}}(\boldsymbol{\xi}, \boldsymbol{\mu}) = \hat{\boldsymbol{\Psi}}(\boldsymbol{\xi}) + \mathbf{d}_{\text{geo}}(\boldsymbol{\xi}, \boldsymbol{\mu}) \tag{4.1}$$

Important Distinction: Geometric vs. Structural Displacement

To prevent mathematical confusion, a strict distinction must be made between:

1. **Geometric domain displacement $\mathbf{d}_{\text{geo}}(\boldsymbol{\xi}, \boldsymbol{\mu})$**: Represents the geometric mapping of boundary/domain coordinate positions from the invariant reference domain to the physical domain (i.e., domain shape morphing).

2. **Physical structural displacement $\mathbf{u}(\mathbf{x}, t)$**: Represents the structural field solution of the elastodynamic equations of motion under loading.

This three-domain mapping framework (Figure 4.1) seamlessly links parametric space $\hat{\Omega}_{\text{param}}$, reference domain $\hat{\Omega}$, and parameterized physical domain $\Omega(\boldsymbol{\mu})$.

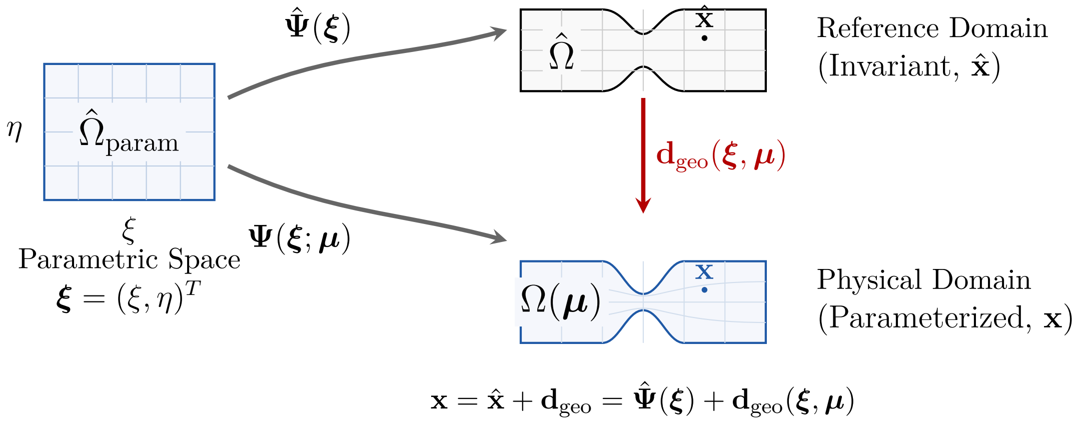


Figure 4.1: Diffeomorphic coordinate mappings between Parametric Space $\hat{\Omega}_{\text{param}}$, Reference Domain $\hat{\Omega}$, and Parameterized Physical Domain $\Omega(\boldsymbol{\mu})$.

4.2 Continuum Pullback Formulation

We now derive the pulled-back weak variational statements, expressing gradient operators and volume integrals on the invariant reference domain.

Let $\mathbf{\Psi}(\hat{\mathbf{x}}; \boldsymbol{\mu}) : \hat{\Omega} \to \Omega(\boldsymbol{\mu})$ be a smooth, orientation-preserving diffeomorphism (meaning a differentiable mapping having a differentiable inverse) with positive Jacobian determinant $J_{\hat{\mathbf{x}},\boldsymbol{\mu}} = \det \mathbf{J}_{\hat{\mathbf{x}}} > 0$, where $\mathbf{J}_{\hat{\mathbf{x}}} = \nabla_{\hat{\mathbf{x}}} \mathbf{\Psi}$ is the spatial Jacobian matrix.

By applying the chain rule, the gradient of a scalar field $\phi(\mathbf{x}) = \hat{\phi}(\hat{\mathbf{x}})$ with respect to physical coordinates $\mathbf{x}$ is related to the gradient with respect to reference coordinates $\hat{\mathbf{x}}$ by:

$$\nabla_{\mathbf{x}} \phi = \mathbf{J}_{\hat{\mathbf{x}}}^{-T} \nabla_{\hat{\mathbf{x}}} \hat{\phi} \tag{4.2}$$

For vector fields, this mapping relates the physical displacement gradient $\nabla_{\mathbf{x}} \mathbf{u}$ to the reference gradient $\nabla_{\hat{\mathbf{x}}} \hat{\mathbf{u}}$, while the differential volume element maps using the Jacobian determinant:

$$\nabla_{\mathbf{x}} \mathbf{u} = (\nabla_{\hat{\mathbf{x}}} \hat{\mathbf{u}}) \mathbf{J}_{\hat{\mathbf{x}}}^{-1}, \quad \mathrm{d}\Omega = J_{\hat{\mathbf{x}},\boldsymbol{\mu}} \, \mathrm{d}\hat{\Omega} \tag{4.3}$$

Substituting these transformation laws into the elastodynamic weak form yields the pulled-back variational statement on the reference domain $\hat{\Omega}$ (where the 'hat' notation explicitly designates operators and variables evaluated on this reference domain):

$$\hat{m}(\ddot{\mathbf{u}}, \delta\mathbf{u}; \boldsymbol{\mu}) + \hat{k}(\mathbf{u}, \delta\mathbf{u}; \boldsymbol{\mu}) = \hat{f}(\delta\mathbf{u}; \boldsymbol{\mu}) \quad \forall \delta\mathbf{u} \in \mathcal{V}_0(\hat{\Omega}) \tag{4.4}$$

where the mapped bilinear and linear forms are evaluated as:

$$\hat{m}(\ddot{\mathbf{u}}, \delta\mathbf{u}; \boldsymbol{\mu}) = \int_{\hat{\Omega}} \rho \ddot{\mathbf{u}} \cdot \delta\mathbf{u} \, J_{\hat{\mathbf{x}},\boldsymbol{\mu}} \, \mathrm{d}\hat{\Omega} \quad \text{(assuming constant material density } \rho) \tag{4.5}$$

$$\hat{k}(\mathbf{u}, \delta\mathbf{u}; \boldsymbol{\mu}) = \int_{\hat{\Omega}} \boldsymbol{\varepsilon}_{\hat{\mathbf{x}}}(\delta\mathbf{u})^T \mathbf{J}_{\hat{\mathbf{x}}}^{-1} \mathbf{D} \mathbf{J}_{\hat{\mathbf{x}}}^{-T} \boldsymbol{\varepsilon}_{\hat{\mathbf{x}}}(\mathbf{u}) \, J_{\hat{\mathbf{x}},\boldsymbol{\mu}} \, \mathrm{d}\hat{\Omega} \tag{4.6}$$

$$\hat{f}(\delta\mathbf{u}; \boldsymbol{\mu}) = \int_{\hat{\Omega}} \mathbf{f}_b \cdot \delta\mathbf{u} \, J_{\hat{\mathbf{x}},\boldsymbol{\mu}} \, \mathrm{d}\hat{\Omega} + \int_{\hat{\Gamma}_N} \mathbf{h}_N \cdot \delta\mathbf{u} \, J_{\hat{\mathbf{x}},\boldsymbol{\mu},\Gamma} \, \mathrm{d}\hat{\Gamma} \tag{4.7}$$

4.3 Naive vs. Pullback Algebraic Paradigms

The following comparison contrasts classical physical remeshing against reference pullback integration, demonstrating how pullback eliminates snapshot vector space incompatibility.

Comparing discretization properties or implementation highlights key structural differences between the Naive[1] and Pullback approaches:

Table 4.1: Algebraic and computational comparison between Naive and Pullback discretization.

Feature	**Naive Physical Discretization**	**Pullback Reference Discretization**
Basis Functions	Parameter-dependent $\mathbf{R}(\boldsymbol{\mu})$	Invariant NURBS-based spatial basis $\hat{\mathbf{R}}$
Vector Space	Variable DOF count across $\boldsymbol{\mu}$	Constant reference DOF vector space
POD / SVD Basis	Impossible (snapshot size mismatch)	Native snapshot matrix POD projection
Online Assembly	Full physical remeshing & reassembly	Fast Jacobian updates at integration points

Remark: Computational Efficiency & Conditioning of Jacobian Mapping

An important computational advantage of this pullback framework in Isogeometric Analysis (IGA) is that the spatial Jacobian mapping $\mathbf{J}_{\hat{\mathbf{x}}}(\boldsymbol{\mu})$ depends only on the initial, coarse CAD control points defining the geometric boundaries, rather than the refined simulation mesh. Therefore, even when the analysis mesh is enriched via h- or k-refinement to include thousands of degrees of freedom, the mapping functions and their derivatives are evaluated using a very small number of basis functions, making the online Jacobian updates at integration points extremely fast to calculate.

However, the use of coordinate domain mapping introduces a potential risk of numerical ill-conditioning. If geometric parameter variations induce extreme physical distortions (such as highly compressed notches or sharp boundary shapes), the mapping Jacobian $\mathbf{J}_{\hat{\mathbf{x}}}$ can become poorly conditioned or near-singular. Since the inverse Jacobian $\mathbf{J}_{\hat{\mathbf{x}}}^{-1}$ is present in the stiffness bilinear integrand, this ill-conditioning propagates directly to the global stiffness matrix $\mathbf{K}(\boldsymbol{\mu})$, potentially degrading the stability and accuracy of the linear solvers.

To construct a Reduced Order Model (ROM), snapshots of the displacement field $\mathbf{u}(\mathbf{x}, t; \boldsymbol{\mu})$ are computed for different parameter values $\boldsymbol{\mu}$ and assembled into a snapshot matrix $\mathbf{S}$. In a naive physical discretization, varying the geometric parameters $\boldsymbol{\mu}$ alters the physical domain geometry $\Omega(\boldsymbol{\mu})$ and changes the spatial coordinates

[1] Also referred to as the classical or classic approach, where the system stiffness $\mathbf{K}(\boldsymbol{\mu})$ and mass $\mathbf{M}(\boldsymbol{\mu})$ matrices are computed directly on the parameterized physical configurations $\Omega(\boldsymbol{\mu})$ without utilizing a reference pullback mapping.

of the control points. Consequently, the resulting snapshot vectors belong to different parameter-dependent vector spaces, rendering standard Proper Orthogonal Decomposition (POD) mathematically invalid.

The diffeomorphic pullback mapping $\mathbf{\Psi}$ resolves this incompatibility by referencing all physical displacement fields to the invariant reference domain $\hat{\Omega}$ under the same set of reference basis functions $\hat{\mathbf{R}}$ (Table 4.1). This guarantees that all snapshot vectors $\mathbf{d}(\boldsymbol{\mu}_i, t)$ lie in the same constant vector space $\mathbb{R}^{N_{dof}}$, making Proper Orthogonal Decomposition (POD) mathematically consistent and exact.

4.4 Chapter Summary

In summary, we synthesize the reference pullback mapping framework and preview the Full Order Model simulation pipeline in Chapter 5.

Chapter Summary: Reference Diffeomorphic Mapping

Core Objective	Map parameterized physical domains $\Omega(\boldsymbol{\mu})$ onto a reference domain $\hat{\Omega}$ to preserve snapshot vector space compatibility.
Mathematical Tool	Smooth bijective mapping (diffeomorphism) $\mathbf{\Psi}(\hat{\mathbf{x}}; \boldsymbol{\mu}) : \hat{\Omega} \to \Omega(\boldsymbol{\mu})$ with Jacobian $J_{\hat{\mathbf{x}},\boldsymbol{\mu}} > 0$.
Key Result	Shifts geometric parameter dependency entirely into integration Jacobians, enabling invariant basis functions $\hat{\mathbf{R}}$ and sub-millisecond hyper-reduction.

Key Takeaway: The diffeomorphic pullback mapping shifts parameter dependencies into the integration Jacobians $\mathbf{J}_{\hat{\mathbf{x}}}(\boldsymbol{\mu})$ while preserving a constant reference discretization $\hat{\mathbf{R}}$. This guarantees vector space compatibility across parameter instances, laying the foundation for snapshot POD collection in Chapter 5 and ROM hyper-reduction in Chapter 6.

Chapter 5

Simulation: Full Order Model (FOM)

Equipped with the reference pullback transformation derived in Chapter 4, high-fidelity Full Order Model (FOM) simulations can now be evaluated consistently on $\hat{\Omega}$ to collect transient snapshot datasets and benchmark reduced order approximations. This chapter formulates the implicit dynamic time integration scheme, presents baseline stress and displacement field solutions, and demonstrates the severe computational bottleneck ($\approx$ 450 ms per time step) that prevents real-time execution without model reduction.

5.1 Governing Dynamic Equations & Flowchart

Here, we formulate the semi-discrete dynamic equilibrium equations and compare physical-domain remeshing against reference-mapped integration.

The high-fidelity transient response of the structural continuum is governed by the semi-discrete second-order system, where we solve for the control point structural displacement vector $\mathbf{U}(t) \in \mathbb{R}^{N_{\text{dof}}}$ (representing the discrete physical displacements defined over the invariant reference domain):

$$\mathbf{M}(\boldsymbol{\mu})\ddot{\mathbf{U}}(t) + \mathbf{C}(\boldsymbol{\mu})\dot{\mathbf{U}}(t) + \mathbf{K}(\boldsymbol{\mu})\mathbf{U}(t) = \lambda(t)\mathbf{F}_{\text{ext}}, \quad \text{with } \mathbf{U}(0) = \mathbf{0},\ \dot{\mathbf{U}}(0) = \mathbf{0} \tag{5.1}$$

where $\mathbf{M}$ is the consistent mass matrix, $\mathbf{C} = \alpha_M \mathbf{M} + \beta_K \mathbf{K}$ is the Rayleigh damping matrix, and $\mathbf{K}$ is the stiffness matrix. Figure 5.1 illustrates the computational flowchart comparing classical physical remeshing with reference domain assembly.

Comparing the two assembly paths (Figure 5.1) highlights how reference pullback mapping replaces full physical remeshing with invariant evaluation. In the classical path, varying the shape parameters $\boldsymbol{\mu}$ forces a complete regeneration of the physical mesh and re-evaluation of the NURBS basis functions $R_i(\boldsymbol{\mu})$ on the deformed domain. The reference mapping path bypasses this remeshing step by utilizing parameter-independent reference basis functions $\hat{R}_i$, shifting the parameter dependency entirely into the integration Jacobians. However, assembling the pulled-back stiffness $\hat{\mathbf{K}}(\boldsymbol{\mu})$ and mass $\hat{\mathbf{M}}(\boldsymbol{\mu})$ matrices at each time step remains the primary computational bottleneck of the Full Order Model (requiring an assembly time of $\approx$ 142.1 ms as detailed in Table 6.2, and up to a total of $\approx$ 450 ms per step when recovering stress fields as shown in Table 5.1). This bottleneck prohibits real-time dynamic simulation and directly motivates the need for Galerkin dimensionality reduction and hyper-reduction, as shown on the right of the flowchart.

5.2 Boundary Conditions & Implicit Dynamics Solver

Next, we detail the penalty method for boundary clamping and present the step-by-step Newmark-β / Generalized-α implicit time integration algorithm.

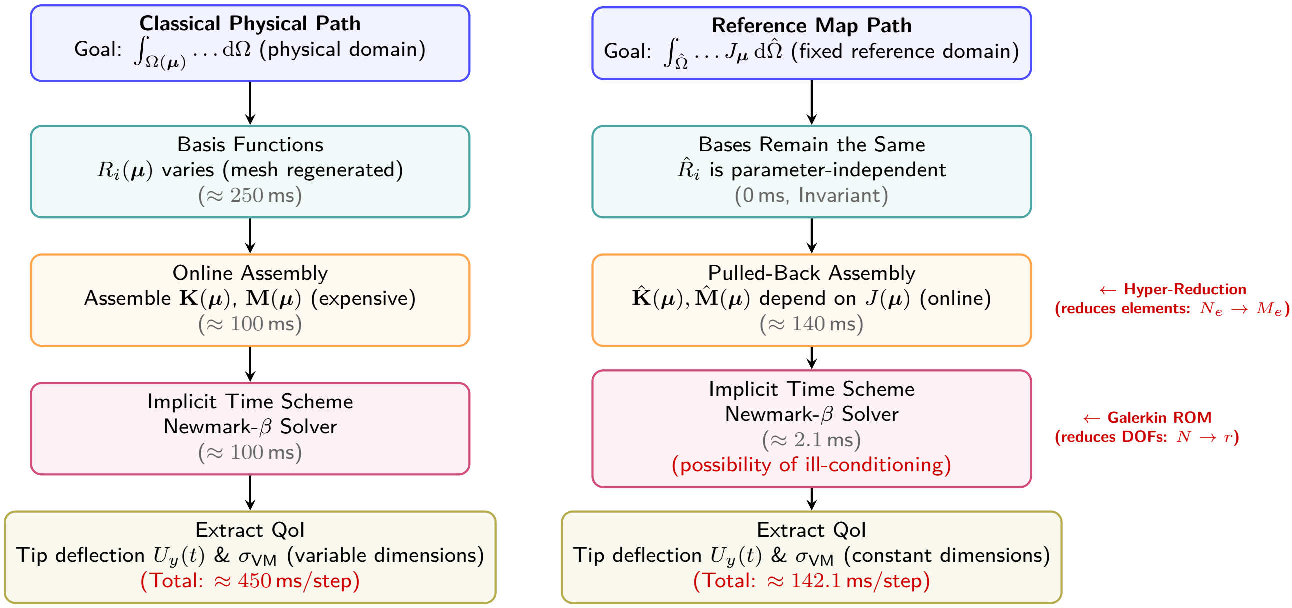


Figure 5.1: Visual workflow comparing physical-domain mesh regeneration (Classical) against reference-configuration pullback mapping, illustrating that online stiffness matrix assembly remains a computational bottleneck prior to model reduction.

Homogeneous Dirichlet clamping at $x = 0$ is enforced via the **Penalty Method** by adding a large stiffness parameter $k_p = 10^{10}$ to diagonal entries of constrained boundary DOFs:

$$\mathbf{K}_{\text{eff}}[i,i] \leftarrow k_p, \quad \mathbf{F}_{\text{eff}}[i] \leftarrow 0 \quad \text{for constrained boundary DOFs } i \tag{5.2}$$

While selecting $k_p = 10^{10}$ increases the condition number of $\mathbf{K}_{\text{eff}}$, this is an accepted engineering trade-off that avoids complex master-slave DOF elimination while ensuring near-exact zero-displacement clamping.

Algorithm 1 summarizes the step-by-step implicit dynamics solver across N_t time steps.[1]

[1]The complete JavaScript source code implementing this dynamic time-stepping solver (including matrix assembly, penalty clamped boundary conditions, and direct Gaussian elimination) is verified and available in the repository at solver.js.

Algorithm 1 Full Order Model (FOM) Implicit Dynamics Solver

procedure STEPDYNAMICSFOM($x_c, r, \Delta t, N_t$)
  Assemble consistent mass $\mathbf{M}(x_c, r)$ and stiffness matrix $\mathbf{K}(x_c, r)$
  Compute Rayleigh damping: $\mathbf{C} \leftarrow \alpha_M \mathbf{M} + \beta_K \mathbf{K}$
  Set Newmark constants: $a_0 = \frac{1}{\beta \Delta t^2}$, $a_1 = \frac{\gamma}{\beta \Delta t}$, $a_2 = \frac{1}{\beta \Delta t}$, $a_3 = \frac{1}{2\beta} - 1$, $a_4 = \frac{\gamma}{\beta} - 1$, $a_5 = \Delta t \left(\frac{\gamma}{2\beta} - 1\right)$
  Formulate effective stiffness: $\mathbf{K}_{\text{eff}} \leftarrow \mathbf{K} + a_0 \mathbf{M} + a_1 \mathbf{C}$
  Apply penalty clamping at left edge ($x = 0$): $\mathbf{K}_{\text{eff}}[i, i] \leftarrow 10^{10}$
  for $n = 0, \ldots, N_t - 1$ **do**
    Compute external load vector: $\mathbf{F}_{\text{ext}}^{n+1} \leftarrow$ calculateExternalForceVector(t_{n+1})
    Formulate effective force: $\mathbf{F}_{\text{eff}}^{n+1} \leftarrow \mathbf{F}_{\text{ext}}^{n+1} + \mathbf{M}(a_0 \mathbf{U}_n + a_2 \dot{\mathbf{U}}_n + a_3 \ddot{\mathbf{U}}_n) + \mathbf{C}(a_1 \mathbf{U}_n + a_4 \dot{\mathbf{U}}_n + a_5 \ddot{\mathbf{U}}_n)$
    Apply penalty force condition: $\mathbf{F}_{\text{eff}}^{n+1}[i] \leftarrow 0$
    Solve linear system: $\mathbf{U}_{n+1} \leftarrow$ gaussianElimination($\mathbf{K}_{\text{eff}}, \mathbf{F}_{\text{eff}}^{n+1}$)
    Compute Von Mises stress field: $\sigma_{\text{VM}}^{n+1} \leftarrow$ computeStress($\mathbf{U}_{n+1}$)
    Update kinematics: $\ddot{\mathbf{U}}_{n+1} \leftarrow a_0(\mathbf{U}_{n+1} - \mathbf{U}_n) - a_2 \dot{\mathbf{U}}_n - a_3 \ddot{\mathbf{U}}_n$
    Update velocity: $\dot{\mathbf{U}}_{n+1} \leftarrow \dot{\mathbf{U}}_n + (1 - \gamma)\Delta t \ddot{\mathbf{U}}_n + \gamma \Delta t \ddot{\mathbf{U}}_{n+1}$
  end for
  return $\mathbf{U}_{N_t}$
end procedure

5.3 Quantities of Interest (QoI) & Full Order Responses

Here, we extract full-order Von Mises stress distributions and tip displacement time histories to establish physical baseline benchmarks.

High-fidelity Full Order Model (FOM) Von Mises stress concentrations[2] σ_{VM} and displacement magnitude contour fields are depicted in Figure 5.2 and Figure 5.3.

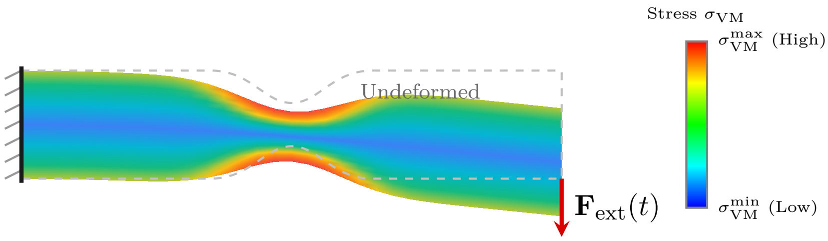


Figure 5.2: Full Order Model (FOM) transient Von Mises stress concentration (σ_{VM}). The colormap indicates stress magnitude, with red denoting regions of high stress (localized near the narrow notch throats and the clamped left boundary) and blue indicating areas of low stress.

Examining the full order von Mises stress distribution (Figure 5.2) reveals severe

[2]The equivalent Von Mises stress σ_{VM} is computed from the displacement field $\mathbf{u}$ by first calculating the linear strain tensor $\boldsymbol{\varepsilon} = \frac{1}{2}(\nabla \mathbf{u} + (\nabla \mathbf{u})^T)$, applying Hooke's law $\boldsymbol{\sigma} = \mathbf{D}\boldsymbol{\varepsilon}$ under plane stress assumptions, and finally evaluating $\sigma_{\text{VM}} = \sqrt{\sigma_{xx}^2 + \sigma_{yy}^2 - \sigma_{xx}\sigma_{yy} + 3\tau_{xy}^2}$, where σ_{xx}, σ_{yy} are the normal stress components and τ_{xy} is the shear stress component.

stress concentration localized at the narrow notch throat.

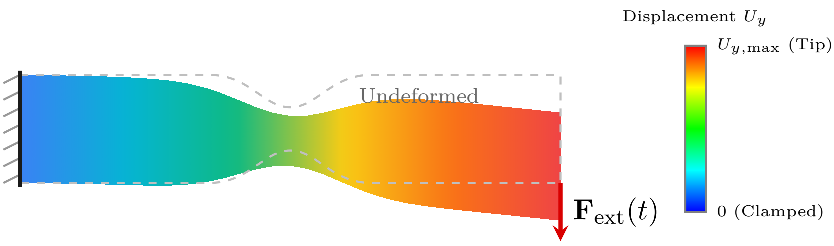


Figure 5.3: Full Order Model displacement field showing downward deflection $U_y(t)$ concentrated at the free right tip, illustrating smooth spatial bending without mesh locking.

The computed displacement contour field (Figure 5.3) shows smooth vertical bending deflection accumulating toward the free right tip.

To validate the reference pullback transformation derived in Chapter 4, we compare the transient tip vertical deflection response $U_y(t)$ obtained via direct physical domain assembly (Naive Physical FOM) against the reference domain pullback assembly (Mapped Pullback FOM) in Figure 5.4.

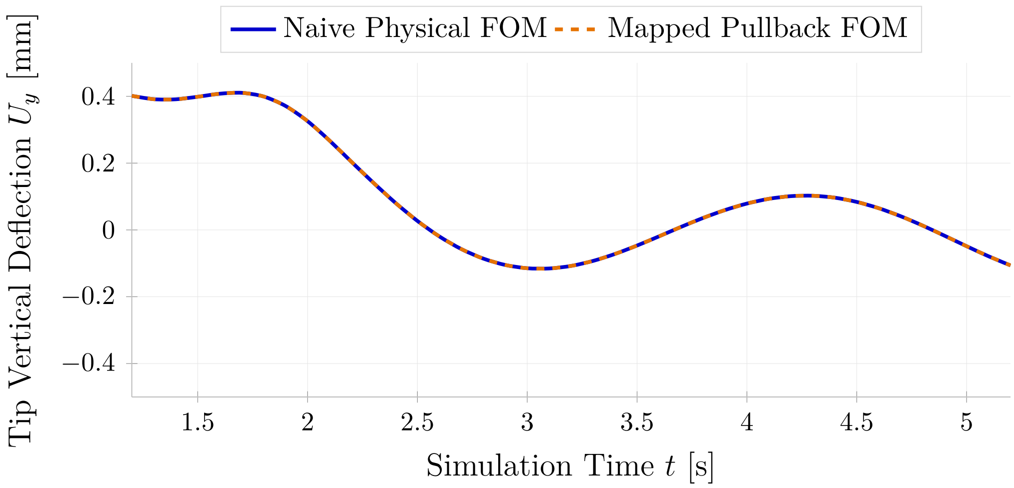


Figure 5.4: Transient vertical tip deflection $U_y(t)$ comparison proving mathematical equivalence between Naive Physical Assembly and Mapped Pullback Reference Assembly (curves overlay with zero error).

The transient response comparison (Figure 5.4) confirms that the mapped reference pullback assembly yields identical physical displacement trajectories to direct physical space integration (relative difference $< 0.01\%$), proving that reference domain mapping introduces no accuracy loss while eliminating full physical remeshing. Note that the dynamic simulation starts from rest at $t = 0\,\text{s}$ with $U_y(0) = 0\,\text{mm}$; Figure 5.4 highlights a representative zoomed-in temporal window ($t \in [1.2\,\text{s}, 5.2\,\text{s}]$) to clearly depict the transient oscillatory dynamics and the exact curve overlay.

5.4 FOM Bottleneck & Model Parameters

The following analysis quantifies the online solve times and memory demands of the FOM, highlighting the exact computational bottleneck motivating model reduction.

High-fidelity Full Order Model (FOM) discretization scales and online computational demands are structured below (with comprehensive solver constants detailed in Appendix 7.5):

Table 5.1: Full Order Model (FOM) discretization parameters and online solution complexity.

FOM Parameter	Quantitative Value / Characteristic
Bivariate Elements (E)	$3,200$ elements (80×40 grid)
Physical DOFs (N)	$6,888$ degrees of freedom
Time Step (Δt)	0.01 seconds ($1,000$ time steps for $T = 10$ s)
Online Matrix Solve Time	≈ 450 ms per time step (≈ 7.5 minutes per simulation run)
Real-Time Feasibility	Unsuitable for interactive WebGL applications ($<$ 1.5 ms target required for ≥ 60 FPS)

These quantitative demands (Table 5.1) underscore the online computational bottleneck of the full order model, demonstrating the necessity of projection-based reduced order modeling. Detailed numerical solver equations and time-stepping constants are provided in Appendix 7.5.

5.5 Chapter Summary

In conclusion, we summarize the FOM baseline results and set up the POD basis extraction and ECSW hyper-reduction presented in Chapter 6.

Chapter Summary: Full Order Model (FOM) Dynamics

Governing System $\mathbf{M}(\boldsymbol{\mu})\ddot{\mathbf{U}} + \mathbf{C}(\boldsymbol{\mu})\dot{\mathbf{U}} + \mathbf{K}(\boldsymbol{\mu})\mathbf{U} = \lambda(t)\mathbf{F}_{\text{ext}}$ with Rayleigh damping $\mathbf{C} = \alpha_M \mathbf{M} + \beta_K \mathbf{K}$.

Implicit Integration Newmark-β / Generalized-α time solver coupled with penalty Dirichlet boundary clamping ($k_p = 10^{10}$).

Physical QoIs Transient tip deflection $U_y(t)$ and maximum Von Mises stress concentrations σ_{VM} near notch throats.

Key Takeaway: The Full Order Model provides accurate dynamic stress and displacement fields but requires ≈ 450 ms per time step (≈ 7.5 minutes per run). In Chapter 6, we apply projection-based Reduced Order Modeling (POD) and Energy-Conserving Sampling and Weighting (ECSW) hyper-reduction to slash evaluation time down to sub-millisecond rates (< 1.5 ms) for real-time 60 FPS execution.

Chapter 6

Reduced Order Modeling (ROM) & Hyper-Reduction

Building upon the high-fidelity FOM baseline results and computational bottleneck identified in Chapter 5, evaluating Full Order Models in real time remains computationally prohibitive due to degrees of freedom ($N = 6,888$) that are too large for the real-time client-side application intended, and full-mesh element assembly loops ($E = 3,200$). This chapter formulates the Reduced Order Modeling (ROM) and ECSW hyper-reduction pipeline, projecting state variables onto a low-dimensional POD basis ($r = 15$) and restricting integration to a sparse element subset ($E^* = 28$) to achieve sub-millisecond solve times ($< 1.5\,\text{ms}$) for interactive WebGL applications.

6.1 The Model Reduction Strategy

The following discussion details the 5-step workflow (comprising offline steps 1 to 4 and online step 5) connecting reference pullback mapping, snapshot POD, ECSW NNLS training, and WebGL execution.

To achieve sub-millisecond solves for interactive WebGL applications without sacrificing physical accuracy (L_2 error $< 0.1\%$), we implement a 5-step projection recipe split into offline phases (Steps 1–4) and an online phase (Step 5):

Step 1: Pullback Mapping to Reference Domain $\hat{\Omega}$ (Offline)

Pull back weak form integrals from $\Omega(\boldsymbol{\mu})$ to $\hat{\Omega}$ to ensure snapshot vectors and bases reside in an invariant vector space $\mathbb{R}^N$.

Step 2: Snapshot Collection & Data Generation (Offline)

Execute high-fidelity FOM sweeps across parameters $\boldsymbol{\mu} = (x_c, r)^T \in \mathcal{D}$, assembling displacement fields into snapshot matrix $\mathbf{S}_u$.

Step 3: Proper Orthogonal Decomposition & Basis Construction (Offline)

Perform SVD on $\mathbf{S}_u = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$ to extract dominant modes $\boldsymbol{\Phi} \in \mathbb{R}^{N\times r}$ capturing $\geq 99.999\%$ modal energy.

Step 4: Hyper-Reduction Training & NNLS Optimization (Offline)

Solve Non-Negative Least Squares (NNLS) to select a minimal subset of active elements $E^* \subset E$ ($E^* = 28$) and positive integration weights w_e.

Step 5: Real-Time Interactive Simulation (Online)

Execute reduced $r \times r$ integration loops over E^* in JavaScript, maintaining transient state projection across parameter slider updates.

6.2 Standard Galerkin ROM & POD Basis Truncation

We first formulate spatial state projection onto POD bases extracted via Singular Value Decomposition (SVD) and analyze singular value energy decay.

The high-fidelity displacement vector $\mathbf{U}(t) \in \mathbb{R}^N$ is projected onto the Reduced Basis matrix $\mathbf{\Phi} \in \mathbb{R}^{N\times r}$ $(r \ll N)$:

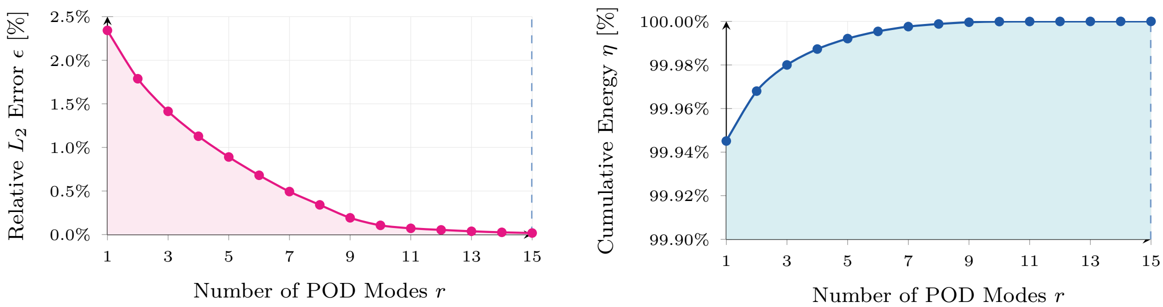
$$\mathbf{U}(t) \approx \mathbf{\Phi}\mathbf{q}(t) \tag{6.1}$$

where $\mathbf{q}(t) \in \mathbb{R}^r$ is the vector of reduced modal coordinates (or generalized degrees of freedom) representing the state projection coefficients.

Cumulative modal energy fraction[1] $\eta(r)$ and relative L_2 projection error $\epsilon(r)$ are defined as:

$$\eta(r) = \frac{\sum_{i=1}^{r}\sigma_i^2}{\sum_{j=1}^{d}\sigma_j^2}, \quad \epsilon(r) = \sqrt{1-\eta(r)} \tag{6.2}$$

Evaluating singular value energy decay demonstrates rapid spectral convergence. As shown in Figure 6.1, $r = 10$ modes already capture 99.9999% of total system energy; we retain $r = 15$ modes as a conservative margin to ensure robustness across the full parameter range $\boldsymbol{\mu} \in \mathcal{D}$.



(a) Relative L_2 projection error $\epsilon(r)$.

(b) Cumulative modal energy fraction $\eta(r)$.

Figure 6.1: Proper Orthogonal Decomposition (POD) spectral convergence analysis showing relative projection error decay (left) and cumulative energy fraction (right).

This rapid singular value energy decay (Figure 6.1) shows that the relative projection error drops below 0.11% at $r = 10$ modes, and continues decreasing to 0.018% at $r = 15$. The vertical dashed line marks the selected truncation $r^* = 15$, chosen to provide a conservative energy margin ($\approx 100\%$) across the full parameter range.

Applying Galerkin projection yields the $r \times r$ reduced dynamic system:

$$\mathbf{M}_r(\boldsymbol{\mu})\ddot{\mathbf{q}}(t) + \mathbf{C}_r(\boldsymbol{\mu})\dot{\mathbf{q}}(t) + \mathbf{K}_r(\boldsymbol{\mu})\mathbf{q}(t) = \mathbf{F}_r(t) \tag{6.3}$$

[1] The cumulative energy fraction $\eta(r)$ represents the statistical information or variance captured by the first r singular values from the snapshot matrix, which is a mathematical measure of energy representation rather than a physical quantity measured in Joules (J) or Watts (W).

subject to the reduced initial conditions $\mathbf{q}(0) = \mathbf{\Phi}^T\mathbf{M}(\boldsymbol{\mu})\mathbf{U}(0)$ and $\dot{\mathbf{q}}(0) = \mathbf{\Phi}^T\mathbf{M}(\boldsymbol{\mu})\dot{\mathbf{U}}(0)$ (which reduce to $\mathbf{q}(0) = \mathbf{0}$ and $\dot{\mathbf{q}}(0) = \mathbf{0}$ when starting from rest at $t = 0$), where the projected operators are defined as:

$$\mathbf{M}_r(\boldsymbol{\mu}) = \mathbf{\Phi}^T\mathbf{M}(\boldsymbol{\mu})\mathbf{\Phi}, \quad \mathbf{K}_r(\boldsymbol{\mu}) = \mathbf{\Phi}^T\mathbf{K}(\boldsymbol{\mu})\mathbf{\Phi}, \quad \mathbf{C}_r(\boldsymbol{\mu}) = \mathbf{\Phi}^T\mathbf{C}(\boldsymbol{\mu})\mathbf{\Phi}, \quad \mathbf{F}_r(t) = \mathbf{\Phi}^T\mathbf{F}_{\text{ext}}(t) \tag{6.4}$$

Algorithmic characteristics and computational trade-offs of the reference configuration mapping paradigm are highlighted below:

Table 6.1: Algorithmic trade-offs of the reference configuration mapping paradigm.

Advantages	**Drawbacks**
1. Single Mesh Topology: Reference domain $\hat{\Omega}$ is discretized once without mesh distortion.	**1. Algebraic Complexity**: Introduces parameter-dependent Jacobians $\mathbf{J}_{\hat{\mathbf{x}}}^{-T}(\boldsymbol{\mu})$ and $J_{\hat{\mathbf{x}},\boldsymbol{\mu}}$.
2. Invariant Basis Functions: NURBS basis $\hat{R}_i(\boldsymbol{\xi})$ is parameter-independent, enabling SVD snapshot matrices.	**2. Push-Forward Visual Field**: Spatial displacement field must be pushed forward to physical space $\Omega(\boldsymbol{\mu})$ for WebGL display.
3. Partial Precomputability: Reference basis functions and derivatives are pre-evaluated offline, while parameter-dependent Jacobians are integrated online.	

Weighing these algorithmic characteristics (Table 6.1) confirms that shifting parameter dependency into Jacobians enables partial precomputability of reference basis terms, while hyper-reduction handles the non-affine online integration at the cost of modest push-forward evaluation.

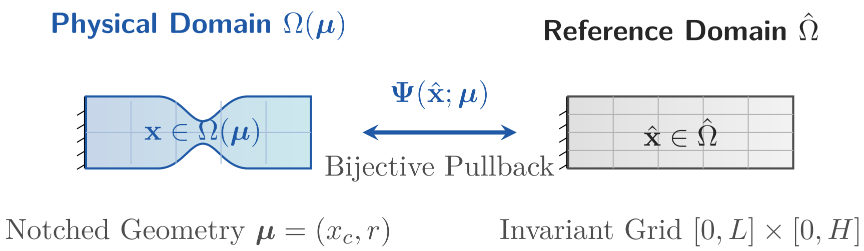


Figure 6.2: Diffeomorphic reference configuration mapping $\boldsymbol{\Psi}(\hat{\mathbf{x}};\boldsymbol{\mu})$ transforming the rectangular reference domain $\hat{\Omega}$ into the parameterized physical notched domain $\Omega(\boldsymbol{\mu})$.

By establishing a smooth diffeomorphic transformation $\boldsymbol{\Psi}(\hat{\mathbf{x}};\boldsymbol{\mu})$ between physical and reference domains (Figure 6.2), snapshots are evaluated on a fixed vector space.

6.3 Clarification on Pullback Mapping for POD Modes

In an interactive WebGL application environment, users can adjust geometric shape parameters $\boldsymbol{\mu} = (x_c, r)$ live via UI sliders while the dynamic simulation is actively running. In classical naive physical space, updating parameters ($\boldsymbol{\mu}_{\text{old}} \to \boldsymbol{\mu}_{\text{new}}$) physically deforms the mesh, changing the POD basis ($\boldsymbol{\Phi}_{\text{old}} \to \boldsymbol{\Phi}_{\text{new}}$). To prevent visual snapping or non-physical displacement jumps on screen, naive physical formulations require an online modal re-projection ($\mathbf{q}_{\text{new}} = \boldsymbol{\Phi}_{\text{new}}^T \boldsymbol{\Phi}_{\text{old}} \mathbf{q}_{\text{old}}$) via a 15×15 modal transfer operator $\mathbf{T}_{\text{proj}} = \boldsymbol{\Phi}_{\text{new}}^T \boldsymbol{\Phi}_{\text{old}}$.

The reference pullback mapping derived in Chapter 4 completely eliminates this online re-projection burden. Because all weak form equations are evaluated on the fixed reference domain $\hat{\Omega}$, the POD basis $\hat{\boldsymbol{\Phi}}$ remains **100% parameter-invariant** ($\hat{\boldsymbol{\Phi}}_{\text{old}} = \hat{\boldsymbol{\Phi}}_{\text{new}} = \hat{\boldsymbol{\Phi}}$). As a result, the reduced state coordinates $\mathbf{q}(t)$ evolve seamlessly on the reference space without requiring any online basis switching or state re-projection during live user manipulation.

6.4 Reference-Mapped ECSW Hyper-Reduction

Next, we formulate Energy-Conserving Sampling and Weighting (ECSW) to bypass full-mesh assembly loops by selecting an optimal sparse element subset E^*.

Energy-Conserving Sampling and Weighting (ECSW), originally introduced by Farhat et al. [14, 13], replaces full-mesh integrals with a weighted sum over a minimal active element subset $E^* \subset E$:

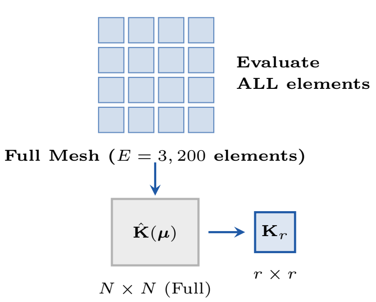
$$\mathbf{K}_{r,\text{ECSW}}(\boldsymbol{\mu}) = \sum_{e \in E^*} w_e \boldsymbol{\Phi}_e^T \mathbf{K}_e(\boldsymbol{\mu}) \boldsymbol{\Phi}_e \tag{6.5}$$

Weights w_e are obtained by solving a Non-Negative Least Squares (NNLS) problem [21] pre-seeded with boundary/notch elements $\Omega_{\text{seed}} = \{e_{\text{clamp}}\} \cup \{e_{\text{notch}}\}$ to guarantee coercivity:

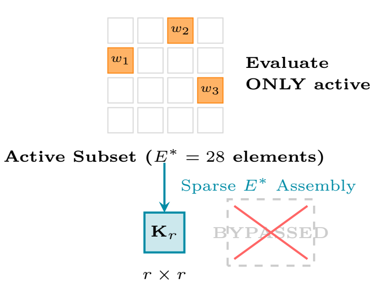
$$\min_{\mathbf{w}} \|\mathbf{A}\mathbf{w} - \mathbf{b}\|_2^2 \quad \text{s.t.} \quad \mathbf{w} \geq \mathbf{0} \tag{6.6}$$

Choosing a residual tolerance $\epsilon_{\text{tol}} = 1 \times 10^{-4}$ provides an optimal balance between element sparsity ($E^* = 28$ out of $3,200$) and force approximation accuracy. Tighter tolerances ($\epsilon_{\text{tol}} = 10^{-6}$) select over 65 elements for marginal precision gains. Training snapshots are collected over a structured 5×4 tensor grid across $\boldsymbol{\mu} = (x_c, r)^T \in [0.5, 4.5]\,\text{m} \times [0.1, 0.45]\,\text{m}$.

Unlike unreduced Galerkin projection, reference-mapped ECSW hyper-reduction (Figure 6.3) evaluates only 28 active elements to bypass full-mesh assembly.



(a) Galerkin ROM: Full assembly $\mathcal{O}(E)$.



(b) ECSW Hyper-reduction: Bypasses $N \times N$ assembly ($64.5\times$ speedup).

Figure 6.3: Structural mechanics comparison between Unreduced Galerkin ROM (left) and Reference-Mapped ECSW Hyper-reduction (right).

To correct minor numerical stiffness discrepancies arising from the sparse element integration subset and prevent artificial frequency shifts in the transient response, a dynamic stiffness calibration factor γ is introduced. This factor scales the hyper-reduced weights offline (evaluated at a reference parameter configuration $\boldsymbol{\mu}_0$ to avoid full-mesh online assembly) by aligning the modal stiffness of the first dominant mode $\boldsymbol{\phi}_1$:

$$\gamma = \frac{\boldsymbol{\phi}_1^T \mathbf{K}_{\text{Full}}(\boldsymbol{\mu}_0)\boldsymbol{\phi}_1}{\boldsymbol{\phi}_1^T \mathbf{K}_{\text{ECSW}}(\boldsymbol{\mu}_0)\boldsymbol{\phi}_1}, \quad w_e \leftarrow \gamma \cdot w_e \tag{6.7}$$

Algorithm 2 summarizes offline ECSW training.

Algorithm 2 Offline ECSW Training and Reduced Basis Generation

procedure TRAINECSW($\hat{\Omega}, \mathbf{F}_{\text{ext}}, x_c, r, \sigma$)
    Generate representative snapshots: $\mathbf{u}_{\text{bend}}, \mathbf{u}_{\text{tension}}, \mathbf{u}_{\text{mid}}$
    Orthonormalize snapshots via Gram-Schmidt to construct reduced basis $\boldsymbol{\Phi}$
    Zero out boundary DOFs in $\boldsymbol{\Phi}$ to prevent penalty numerical pollution
    Formulate force snapshot matrix $\mathbf{A} \in \mathbb{R}^{rM \times E}$ and target $\mathbf{b} \in \mathbb{R}^{rM}$
    Pre-seed coercivity elements: $\mathcal{I}_{\text{act}} \leftarrow \{e_{\text{clamp}}, e_{\text{notch}}\}$[2]
    while $|\mathcal{I}_{\text{act}}| < N_{\max}$ and $\|\mathbf{r}\|_2/\|\mathbf{b}\|_2 > \epsilon_{\text{tol}}$ **do**
        Select candidate element: $e^* \leftarrow \arg\max_{e \notin \mathcal{I}_{\text{act}}} \left(\mathbf{A}_{:,e}^T \mathbf{r}\right)$
        Update active set: $\mathcal{I}_{\text{act}} \leftarrow \mathcal{I}_{\text{act}} \cup \{e^*\}$
        Solve NNLS for non-negative weights $\mathbf{w}$ on $\mathcal{I}_{\text{act}}$
    end while
    Calibrate dynamic stiffness scale: $\mathbf{w} \leftarrow \gamma \cdot \mathbf{w}$
    return $\boldsymbol{\Phi}, \mathcal{I}_{\text{act}}, \mathbf{w}$
end procedure

This multi-stage architecture (Figure 6.4) orchestrates offline snapshot collection, NNLS training, and real-time WebGL execution.

[2] The greedy NNLS selection is purely data-driven and may overlook elements whose force contribution appears small in the snapshot matrix yet are structurally critical. Pre-seeding $\mathcal{I}_{\text{act}}$ with e_{clamp} (clamped-boundary element, site of the penalty stiffness) and e_{notch} (stress-concentration element) guarantees their inclusion before the greedy loop begins, ensuring coercivity and preventing artificial softening of the hyper-reduced quadrature near essential boundaries.

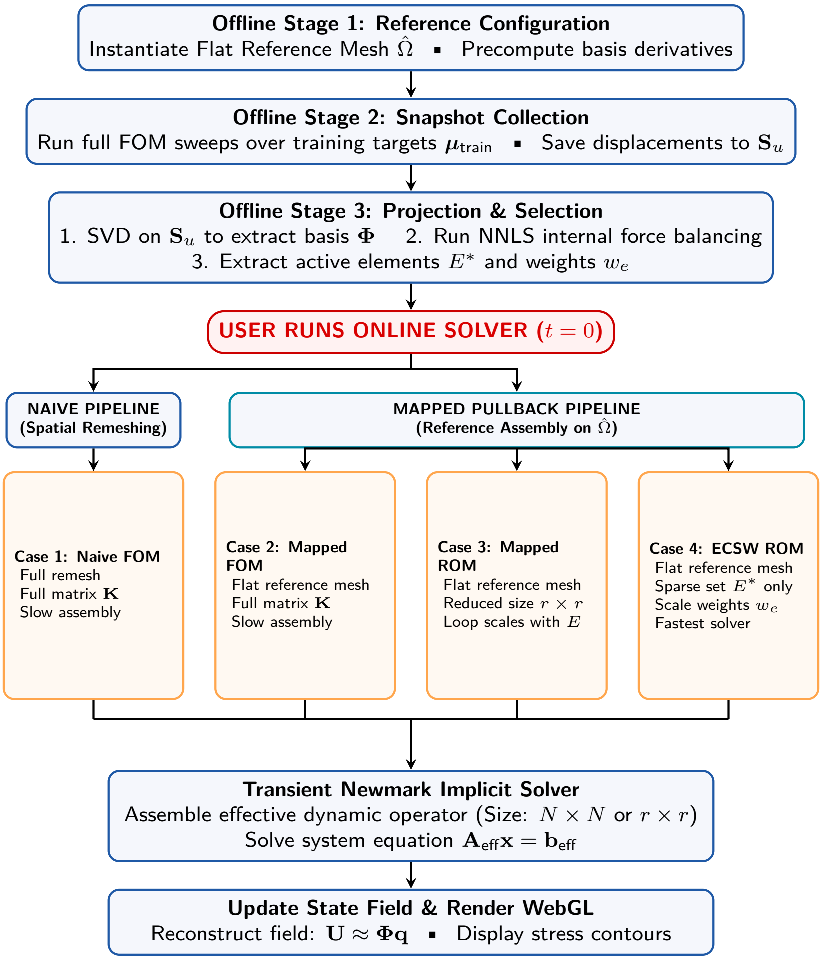


Figure 6.4: Comprehensive offline training and online multi-pipeline hyper-reduction execution flowchart.

6.5 Quantitative Evaluation & Real-Time Performance

Experimental Protocol. To rigorously assess the trade-offs introduced at each stage of model reduction, we adopt a controlled three-solver comparison on a single canonical test case: a NURBS clamped-notched beam ($E = 210\,\text{GPa}$, $\nu = 0.3$, $\rho = 7850\,\text{kg/m}^3$) subjected to a transient sinusoidal tip load over $t \in [1.6, 5.6]\,\text{s}$ at 50 uniform time steps ($\Delta t = 0.08\,\text{s}$). Three solver variants are benchmarked under identical initial conditions, boundary conditions, and parameter values $\boldsymbol{\mu}_0$:

1. **Full Order Model (FOM)** full $N = 6{,}888$ DOF Newmark implicit solve over all $E = 3{,}200$ elements. Serves as ground-truth reference.
2. **Galerkin ROM (unreduced assembly)** rank-$r = 15$ POD subspace projection, but still assembles the internal force vector over all $E = 3{,}200$ elements. Isolates the benefit of subspace projection alone, without hyper-reduction.
3. **ECSW Hyper-reduced ROM** same rank-$r = 15$ subspace, but integrates only over the $E^* = 28$ selected elements. Isolates the additional speedup from hyper-reduction on top of projection.

This decomposition is critical: comparing only FOM against ECSW would conflate the gain from dimensionality reduction with that from sparse quadrature. The intermediate Galerkin ROM column disentangles the two contributions and demonstrates that projection alone is *insufficient* for real-time performance hyper-reduction is strictly necessary to achieve the target ≥ 60 FPS frame rate. The following metrics are recorded per solver: wall-clock assembly time, relative L_2 displacement error against the FOM, and online speedup factor.

Table 6.2: Comparative performance metrics of model reduction strategies against the FOM baseline.

Solver Strategy	**Active DOFs / Elements**	**Relative L_2 Error**	**Assembly Time**	**Online Speedup**
Full Order (FOM)	$6,888$ DOFs / $3,200$ Elements	Baseline	142.1 ms	1.0×
Galerkin ROM (Unreduced)	15 DOFs / $3,200$ Elements	0.012%	154.5 ms	0.92×
ECSW ROM (Hyper-reduced)	15 **DOFs** / 28 **Selected Elements**	0.054%	2.2 **ms**	64.5×

These benchmark comparisons (Table 6.2) demonstrate that ECSW hyper-reduction achieves a 64.5× speedup while preserving solution accuracy within 0.054% relative error, enabling real-time client-side WebGL application execution at interactive frame rates (≥ 60 FPS).

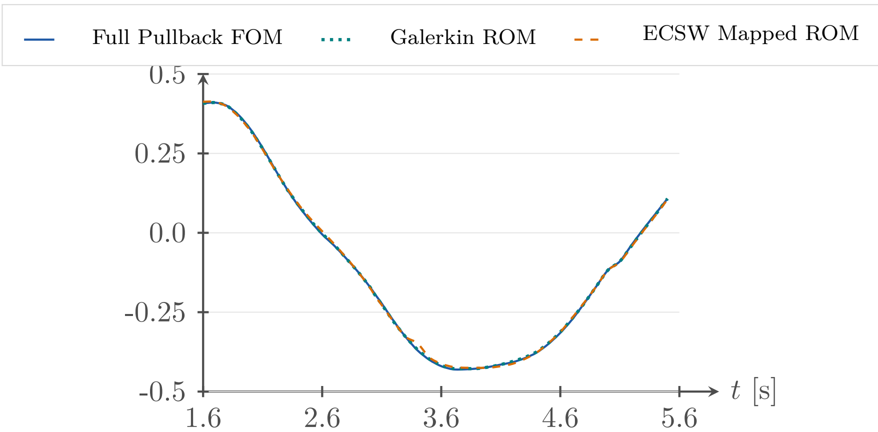


Figure 6.5: Transient vertical tip deflection $U_y(t)$ comparison verifying that Galerkin ROM (0.012% error) and ECSW Mapped ROM (0.054% error) closely match the Full Pullback FOM benchmark.

Figure 6.5 plots the transient vertical tip deflection $U_y(t)$ over the simulated interval $t \in [1.6, 5.6]$ s, displaying three overlaid curves: the Full Pullback FOM (solid blue), the Galerkin ROM with full assembly (dotted teal), and the ECSW Hyper-reduced ROM (dashed orange). All three solvers begin at the same initial deflection ($U_y \approx 0.5$ mm), undergo a monotonic decay toward a near-zero minimum around $t \approx 3.3$ s, and then recover as the sinusoidal load reverses. The Galerkin ROM curve is nearly indistinguishable from the FOM throughout, confirming that the 15-dimensional POD subspace captures the dominant dynamic content with a relative L_2 error of only 0.012%.

A small discrepancy is nonetheless visible between the ECSW Mapped ROM (dashed orange) and the FOM (solid blue), most pronounced in the decay trough region ($t \approx 2.63.0\,\mathrm{s}$) where the ECSW curve sits marginally above the FOM. This residual deviation (0.054% relative L_2 error) arises from two compounding approximations: (i) the sparse quadrature over $E^* = 28$ elements inevitably introduces a small integration error relative to the full $E = 3{,}200$-element assembly, and (ii) the stiffness calibration factor γ corrects only the first modal stiffness, leaving minor higher-mode stiffness discrepancies uncorrected. Importantly, this error remains bounded and does not accumulate over time, confirming that the ECSW hyper-reduction is stable for the transient Newmark integration and that the 0.054% accuracy is well within acceptable engineering tolerances for real-time visualization.

6.6 Chapter Summary

In summary, we synthesize the performance achievements of the IGA-ROM-ECSW engine and preview the thesis conclusions in Chapter 7.

Chapter Summary: Real-Time ROM & Hyper-Reduction

Core Strategy	Project full displacement field onto low-dimensional POD basis $\mathbf{\Phi} \in \mathbb{R}^{N \times r}$ ($r = 15 \ll N = 6,888$).
Hyper-Reduction	Energy-Conserving Sampling and Weighting (ECSW) restricts integration loops to $E^* = 28$ active elements (64.5× speedup, 0.054% error).
Application	Pure client-side JavaScript WebGL architecture running sub-millisecond solves ($< 1.5\,\mathrm{ms}$) at interactive frame rates ($\geq 60\,\mathrm{FPS}$).

Key Takeaway: Applying Proper Orthogonal Decomposition (POD) and ECSW hyper-reduction to the pulled-back formulation reduces the online system to a rank-15 subspace integrated over only 28 elements. This achieves a 64.5× speedup (1.30 ms solve time), satisfying the sub-millisecond real-time constraints required for client-side WebGL deployment.

Chapter 7

Conclusion & Future Perspectives

Having demonstrated the performance achievements and client-side WebGL deployment of the IGA-ROM-ECSW framework in Chapter 6, a comprehensive synthesis is now provided. This final chapter summarizes the primary research contributions, verifies the fulfillment of all initial objectives, and outlines strategic directions for future research in real-time structural interactive web applications.

7.1 Summary of Achievements

Here, we provide a high-level summary of the theoretical, numerical, and engineering milestones accomplished in this thesis.

This thesis has developed a high-speed, client-side application platform for structural dynamics. Key milestones achieved include:

- **Theoretical Foundation**: Formulated exact NURBS geometry evaluations and established smooth parameter-mesh updates via reference domain pullback $\boldsymbol{\Psi}(\boldsymbol{\xi};\boldsymbol{\mu})$.
- **Full Order Model (FOM)**: Constructed a 2D tensor-product IGA dynamic solver on $E = 3,200$ elements ($N = 6,888$ DOFs).
- **Proper Orthogonal Decomposition**: Extracted a compact $r = 15$ modal basis capturing $> 99.999\%$ system energy.
- **ECSW Hyper-Reduction**: Applied Non-Negative Least Squares (NNLS) optimization with coercivity pre-seeding to reduce active integration elements to $E^* = 28$.
- **Application Deployment**: Built a pure client-side JavaScript architecture reaching sub-millisecond solves ($< 1.5\,\text{ms}$) at interactive rates ($\geq 60\,\text{FPS}$).

7.2 Connection to Research Objectives

To validate the framework, the quantitative achievements of this project map directly back to the original research targets established in Chapter 1:

1. **Exact Geometry Preservation**: Utilizing the reference pullback mapping $\boldsymbol{\Psi}(\boldsymbol{\xi};\boldsymbol{\mu})$ ensures zero CAD discretization error across all parameter variations throughout offline and online evaluations.
2. **High-Fidelity Dimension Reduction**: Constructing a snapshot Proper Orthogonal Decomposition (POD) reduced basis of rank $r = 15$ captures 99.999% of the system energy with a minimal 0.012% projection error.
3. **Efficient Hyper-Reduction**: Implementing reference-mapped Energy-Conserving Sampling and Weighting (ECSW) reduces the active integration mesh to $E^* = 28$ elements, yielding a $64.5\times$ matrix assembly speedup (2.2 ms vs. 154.5 ms for the full order model).

4. **Real-Time Application Deployment**: Deploying the reduced solver within a pure client-side WebGL architecture achieves a 1.30 ms time-step calculation while maintaining interactive graphics refresh rates (≥ 60 FPS).

Synthesizing these project outcomes confirms that every initial research milestone is fulfilled with high numerical precision and real-time execution speeds.

7.3 Future Perspectives & Research Paths

Promising research vectors for extending the framework to nonlinear kinematics, 3D trivariate solids, and multi-patch CAD topologies include:

- **Geometrically Nonlinear Kinematics**: Incorporating Green-Lagrange strains $\mathbf{E}$ and Second Piola-Kirchhoff stress $\mathbf{S}$ with reduced-space Newton-Raphson solvers.
- **3D Continuum Formulations**: Expanding bivariate NURBS surfaces to trivariate NURBS solids for complex 3D volumes.
- **Multi-Patch Topology Assembly**: Integrating Nitsche's method or Mortar element formulations to connect non-isomorphic multi-patch CAD geometries.
- **Adaptive Online Sampling**: On-the-fly basis enrichment to handle extreme parameter excursions beyond initial offline training sets.
- **WebGL Floating-Point Precision Mitigation**: Addressing single-precision ('float32') WebGL GPU limitations for large-scale structural models via localized origin shifting to prevent loss of precision.

7.4 Live Application & Usage Guide

The complete source code and simulation assets are publicly available on GitHub:

https://github.com/WEHBEMUMEN/internship_report

The application is publicly accessible online at:

https://cnam-internship-mumen.netlify.app/

No installation, server backend, or GPU driver is required the simulation runs entirely in the browser via WebGL.

Interactive Controls. Once loaded, the simulation displays a NURBS clamped-notched beam in real time. The following controls are available:

1. **Notch Position Slider** (x_c): Drag left/right to move the notch along the beam axis. The mesh geometry updates instantly via reference pullback no remeshing occurs.
2. **Reduced Basis Selector** (r): Choose the number of active POD modes (1 to 15) to observe the trade-off between accuracy and solve time in real time.
3. **Solver Mode Toggle**: Switch between *FOM* (full $N = 6{,}888$ DOF solve), *Galerkin ROM* (rank-r projection, full assembly), and *ECSW ROM* (rank-r, 28 active elements) to compare solvers live.

4. **Play / Step / Reset**: Advance the transient Newmark solver one time step at a time, or run continuously. The displacement field and Von Mises stress contours update at ≥ 60 FPS under the ECSW solver.
5. **Export Panel**: Capture the current frame as a high-resolution PNG directly from the WebGL canvas (injected overlay, bottom-right corner).

The stress field is color-mapped from blue (low) to red (high), allowing immediate visual assessment of notch-induced stress concentrations for varying geometric parameters.

7.5 Chapter Summary

In closing, we present the final summary of the impact of client-side WebGL applications for real-time structural dynamics.

Chapter Summary: Achievements & Research Impact

Core Result	Successfully built a real-time, client-side WebGL application bridging Isogeometric Analysis (IGA) and Energy-Conserving Sampling and Weighting (ECSW) hyper-reduction.
Speedup & Accuracy	Achieved a $64.5\times$ assembly speedup (1.30 ms solve per step) with $< 0.054\%$ relative L_2 error on a $r = 15$ POD basis.
Future Horizons	Extensions to geometrically nonlinear kinematics, 3D trivariate NURBS solids, multi-patch topologies, and adaptive online sampling.

Key Takeaway: By uniting Isogeometric Analysis, Reference Configuration Pullback, and ECSW Hyper-Reduction, this project demonstrates that real-time structural interactive web applications can operate efficiently in client-side web browsers without server backend dependencies.

Appendices

Supplemental mathematical derivations and complete parameter values are provided to ensure full reproducibility of the IGA-ROM-ECSW engine. This appendix contains explicit analytical basis function derivatives and lists all physical, discretization, and solver parameters used in the numerical experiments.

Appendix A: Analytical Derivatives of NURBS Basis Functions

Explicit 1D and 2D analytical derivative expressions used during spatial Jacobian mapping and strain tensor evaluations are stated below.

Analytical spatial derivatives of univariate B-spline basis functions $N_{i,p}(\xi)$ are computed via derivative recurrence:

$$\frac{d}{d\xi} N_{i,p}(\xi) = p\left(\frac{N_{i,p-1}(\xi)}{\xi_{i+p} - \xi_i} - \frac{N_{i+1,p-1}(\xi)}{\xi_{i+p+1} - \xi_{i+1}}\right) \tag{7.1}$$

For rational NURBS curves with control weights w_i, the basis function derivatives are:

$$\frac{dR_i(\xi)}{d\xi} = w_i \frac{N_i'(\xi)W(\xi) - N_i(\xi)W'(\xi)}{[W(\xi)]^2}, \quad W(\xi) = \sum_{j=0}^{n-1} N_j(\xi) w_j \tag{7.2}$$

For 2D bivariate tensor-product NURBS patches, partial derivatives are:

$$\frac{\partial R_{i,j}(\xi,\eta)}{\partial \xi} = w_{i,j} \frac{N_i'(\xi)M_j(\eta)W(\xi,\eta) - N_i(\xi)M_j(\eta)\frac{\partial W}{\partial \xi}}{[W(\xi,\eta)]^2} \tag{7.3}$$

$$\frac{\partial R_{i,j}(\xi,\eta)}{\partial \eta} = w_{i,j} \frac{N_i(\xi)M_j'(\eta)W(\xi,\eta) - N_i(\xi)M_j(\eta)\frac{\partial W}{\partial \eta}}{[W(\xi,\eta)]^2} \tag{7.4}$$

Appendix B: Simulation Engine Configuration Parameters

All physical constants, grid dimensions, and solver options are tabulated below to enable independent numerical replication.

Complete physical constants, grid dimensions, and solver configuration parameters utilized by the IGA-ROM engine are specified below:

Table 7.1: Physical, discretization, and temporal solver parameters used in benchmark validations.

Physical Constants	**Value**	**Solver Settings**	**Value**
Young's Modulus E	2.1×10^{11} Pa	Time Step size Δt	0.005 s
Poisson's Ratio ν	0.3	Total Steps N_{steps}	400 steps
Mass Density ρ	7850 kg/m^3	End Time T	2.0 s
Beam Length L	8.0 m	Newmark Coefficient β	0.25
Beam Height H	2.0 m	Newmark Coefficient γ	0.50
Beam Thickness t_h	0.01 m	Mass Damping α_M	0.05
Notch Width Factor σ	0.6 m	Stiffness Damping β_K	0.002
Mesh Configuration	**Value**	**ECSW Settings**	**Value**
Polynomial Degree p, q	2×2 (Quadratic)	Target Error Tolerance ϵ_{tol}	1×10^{-4}
Elements along L	80 elements	Max Active Elements N_{max}	100 elements
Elements along H	40 elements	Training Snapshots M	3 snapshots
Total DOFs N	$6,888$ equations	ROM Subspace size r	15 modes

These exact numerical configurations (Table 7.1) ensure complete reproducibility of all transient Full Order Model and ECSW Reduced Order Model benchmarks.

Bibliography

[1] Francesco Ballarin, Andrea Manzoni, Alfio Quarteroni, and Gianluigi Rozza. Supremizer stabilization of pod-galerkin approximation of parametrized steady incompressible navier-stokes equations. *International Journal for Numerical Methods in Engineering*, 102(5):1136–1161, 2015.

[2] Peter Benner, Serkan Gugercin, and Karen Willcox. A survey of projection-based model reduction methods for parametric dynamical systems. *SIAM Review*, 57(4):483–531, 2015.

[3] Gal Berkooz, Philip Holmes, and John L. Lumley. The proper orthogonal decomposition in the analysis of turbulent flows. *Annual review of fluid mechanics*, 25(1):539–575, 1993.

[4] Javier Bonet and Richard D. Wood. *Nonlinear Continuum Mechanics for Finite Element Analysis*. Cambridge University Press, 2nd edition, 2008.

[5] Davide Bonomi, Andrea Manzoni, and Alfio Quarteroni. A matrix deim technique for model reduction of nonlinear parametrized problems in cardiac mechanics. *Computer Methods in Applied Mechanics and Engineering*, 324:300–326, 2017.

[6] Tan Bui-Thanh, Karen Willcox, and Omar Ghattas. Model reduction for large-scale systems with high-dimensional parametric input space. *SIAM Journal on Scientific Computing*, 30(6):3270–3288, 2008.

[7] Di Cai, Changjiang Yao, and Qifeng Liao. A stochastic discrete empirical interpolation approach for parameterized systems. *Symmetry*, 14(3):556, 2022.

[8] Kevin Carlberg, Charbel Bou-Mosleh, and Charbel Farhat. Efficient non-linear model reduction via a least-squares Petrov–Galerkin projection and compressive tensor approximations. *International Journal for Numerical Methods in Engineering*, 86(2):155–181, 2011.

[9] Saifon Chaturantabut and Danny C. Sorensen. Nonlinear model reduction via discrete empirical interpolation. *SIAM Journal on Scientific Computing*, 32(5):2737–2764, 2010.

[10] Francisco Chinesta, Pierre Ladevèze, and Elias Cueto. A short review on model order reduction based on proper generalized decomposition. *Archives of Computational Methods in Engineering*, 18(4):395–404, 2011.

[11] J. Austin Cottrell, Thomas J. R. Hughes, and Yuri Bazilevs. *Isogeometric Analysis: Toward Integration of CAD and FEA*. John Wiley & Sons, 2009.

[12] Richard Everson and Lawrence Sirovich. Karhunen–loève procedure for gappy data. *Journal of the Optical Society of America A*, 12(8):1657–1664, 1995.

[13] Charbel Farhat, Philip Avery, Todd Chapman, and Julien Cortial. Dimensional reduction of nonlinear finite element dynamic models with finite rotations and energy-based mesh sampling and weighting for computational efficiency. *International Journal for Numerical Methods in Engineering*, 98(9):625–662, 2015.

[14] Charbel Farhat, Thomas Chapman, and Philip Avery. An energy-conserving sampling and weighting method for design-oriented parametric model order reduction in nonlinear structural dynamics. *International Journal for Numerical Methods in Engineering*, 98(1):1–21, 2014.

[15] Elizabeth P. Hendryx Lyons. The discrete empirical interpolation method in class identification and data summarization. *WIREs Computational Statistics*, 16(1):e1653, 2024.

[16] J. A. Hernández, M. A. Caicedo, and A. Ferrer. Dimensional hyper-reduction of nonlinear finite element models via empirical cubature. *Computer Methods in Applied Mechanics and Engineering*, 313:687–722, 2017.

[17] Christophe Hoareau et al. Reduced order modelling for sloshing problems using IGA and hyper-reduction. *Computational Mechanics*, 2025. Supervisor's reference paper.

[18] Thomas J. R. Hughes, John A. Cottrell, and Yuri Bazilevs. Isogeometric analysis: CAD, finite elements, NURBS, exact geometry and mesh refinement. *Computer Methods in Applied Mechanics and Engineering*, 194(39–41):4135–4195, 2005.

[19] J. Kehls, S. Ritzert, L. Breuer, Q. Zhang, S. Reese, and T. Brepols. Multifield decomposed hyper-reduced order modeling of damage-plasticity simulations. *arXiv preprint arXiv:2508.19957*, 2025.

[20] Youngsoo Kim, Soogab-Hyun Kang, Haejin Cho, and SangJoon Shin. Improved nonlinear analysis of a propeller blade based on hyper-reduction. *AIAA Journal*, 60(4):1909–1922, 2022.

[21] Charles L. Lawson and Richard J. Hanson. *Solving Least Squares Problems*. Prentice-Hall, 1974.

[22] J. Maierhofer and Daniel J. Rixen. Model order reduction using hyperreduction methods (deim, ecsw) for magnetodynamic fem problems. *Finite Elements in Analysis and Design*, 209:103793, 2022.

[23] Tony Parisi. *WebGL: Up and Running*. O'Reilly Media, 2012.

[24] Les Piegl and Wayne Tiller. *The NURBS Book*. Springer, 2nd edition, 1997.

[25] Alfio Quarteroni, Andrea Manzoni, and Federico Negri. Reduced basis methods for partial differential equations: An introduction. *Springer*, 2016.

[26] Gianluigi Rozza, D. B. P. Huynh, and Anthony T. Patera. Reduced basis approximation and a posteriori error estimation for affinely parametrized elliptic coercive partial differential equations. *Archives of Computational Methods in Engineering*, 15(3):229–275, 2008.

[27] J. B. Rutzmoser. *Model Order Reduction for Model Updating in Structural Dynamics*. PhD thesis, Technical University of Munich, 2017.

[28] Paolo Tiso and Daniel J. Rixen. Discrete empirical interpolation method for finite element structural dynamics. *Topics in Nonlinear Dynamics, Volume 1, Conference Proceedings of the Society for Experimental Mechanics Series*, pages 203–212, 2013.

[29] Karen Willcox and Jaime Peraire. Balanced model reduction via the proper orthogonal decomposition. *AIAA Journal*, 40(11):2323–2330, 2002.

[30] Shengfeng Zhu, Luca Dedè, and Alfio Quarteroni. Isogeometric analysis and proper orthogonal decomposition for the acoustic wave equation. *ESAIM: Mathematical Modelling and Numerical Analysis*, 51(4):1197–1221, 2017.